

Index

S.No	Expt. No.	Name of the Experiment	Date	Sheet No.	Remarks
1	EXERCISE 1	Basics			
2	EXERCISE 2	Importance of Visualizations			
3	EXERCISE 3	Plots Using Basic R			
4	EXERCISE 4	ggplot2 in R			
5	EXERCISE 5	ggplot2 in R			
6	EXERCISE 6	Data Visualization			
7	EXERCISE 7	Working with Sheets			
8	EXERCISE 8	Calculations			
9	EXERCISE 9	Graphs for Analytics			
10	EXERCISE 10	Normal Distribution Variation			
11	EXERCISE 11	Reporting and Visualizing Variation			
12	EXERCISE 12	Depicting Changes Over Time			
13	EXERCISE 13	Reporting			

Introduction

Data Visualization Tools:

- **Tableau:** Powerful for creating interactive dashboards and visualizations, supports various data sources.



- **Microsoft Power BI:** Integrates well with Microsoft products, offers robust visualization and BI capabilities.



- **Matplotlib:** A Python 2D plotting library that produces publication-quality figures in various formats.



- **Seaborn:** Built on top of Matplotlib, it provides a high-level interface for drawing attractive and informative statistical graphics.



- **ggplot2:** A plotting system for R, based on the grammar of graphics, which allows users to create complex visualizations.



- **Excel:** While primarily a spreadsheet tool, Excel offers basic data visualization capabilities with charts and graphs.



- **Datawrapper:** A tool for creating simple, embeddable charts and maps online, suitable for journalists and publishers.

Datawrapper

- **Google Charts:** A versatile web-based tool by Google for creating interactive and customizable charts and graphs, offering a wide range of chart types, easy integration with web applications, and robust customization options to visualize data effectively.



BASICS OF R PROGRAMMING LANGUAGE

R is a vast and open source programming language.

R continues to enjoy audience because it is:

- Multi-platform
- Open-source
- Lightweight

A number of basic concepts of R language include-

- Object and Environment
- Console
- Script
- Operators
- Data Structures
- Packages and libraries
- Shortcuts

Overview of visualization:

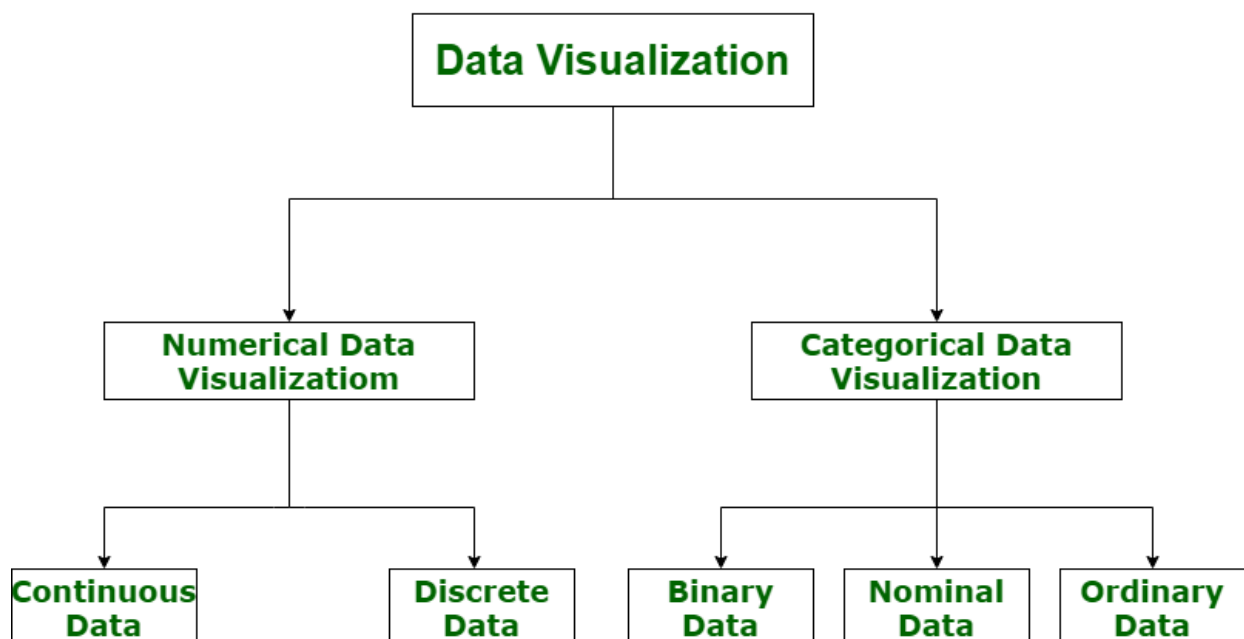
Data visualization is the representation of data through use of common graphics, such as charts, plots, infographics and even animations. These visual displays of information communicate complex data relationships and data-driven insights in a way that is easy to understand. The importance of data visualization is simple: it helps people see, interact with, and better understand data.

This practice is crucial in the data science process, as it helps to make data more understandable and actionable for a wide range of users, from business professionals to data scientists.

Types of Data for Visualization:

Performing accurate visualization of data is very critical to market research where both numerical and categorical data can be visualized, which helps increase the impact of insights and also helps in reducing the risk of analysis paralysis. So, data visualization is categorized into the following categories:

- Numerical Data
- Categorical Data



Graph Plotting in R Programming:

The R programming language provides some easy and quick tools that let us convert our data into visually insightful elements like graphs.

Graph plotting is of two types:

- **One-dimensional Plotting:** In one-dimensional plotting, we plot one variable at a time. For example, we may plot a variable with the number of times each of its values occurred in the entire dataset (frequency). So, it is not compared to any other variable of the dataset. These are the 4 major types of graphs that are used for One-dimensional analysis –
 - Five Point Summary
 - Box Plotting
 - Histograms
 - Bar Plotting
- **Two-dimensional Plotting:** In two-dimensional plotting, we visualize and compare one variable with respect to the other. For example, in a dataset of Air Quality measures, we would like to compare how the AQI varies with the temperature at a particular place. So, temperature and AQI are two different variables and we wish to see how one changes with respect to the other. These are the 3 major kinds of graphs used for such kinds of analysis –
 - Box Plotting
 - Histograms
 - Scatter plots

Different types of graphs:

(1) Bar Graphs:

A bar chart also known as bar graph is a pictorial representation of data that presents categorical data with rectangular bars with heights or lengths proportional to the values that they represent.

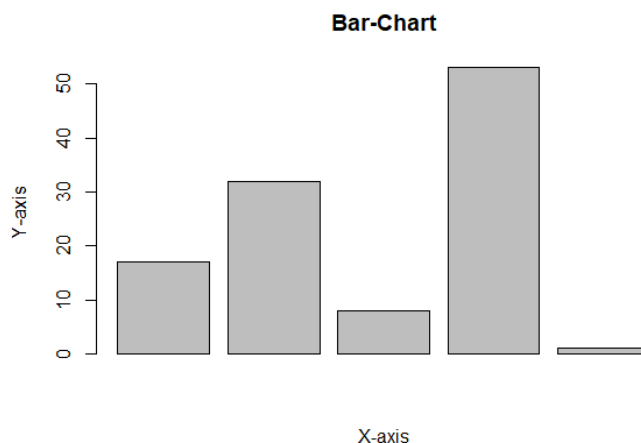
Syntax: `barplot(H, xlab, ylab, main, names.arg, col)`

- **H:** This parameter is a vector or matrix containing numeric values which are used in bar chart.
- **xlab:** This parameter is the label for x axis in bar chart.
- **ylab:** This parameter is the label for y axis in bar chart.
- **main:** This parameter is the title of the bar chart.
- **names.arg:** This parameter is a vector of names appearing under each bar in bar chart.
- **col:** This parameter is used to give colors to the bars in the graph.

Ex:

```
A <- c(17, 32, 8, 53, 1)
```

```
barplot(A, xlab = "X-axis", ylab = "Y-axis", main = "Bar-Chart")
```



(2)Box Plot:

A box graph is a chart that is used to display information in the form of distribution by drawing boxplots for each of them. This distribution of data is based on five sets (minimum, first quartile, median, third quartile, and maximum).

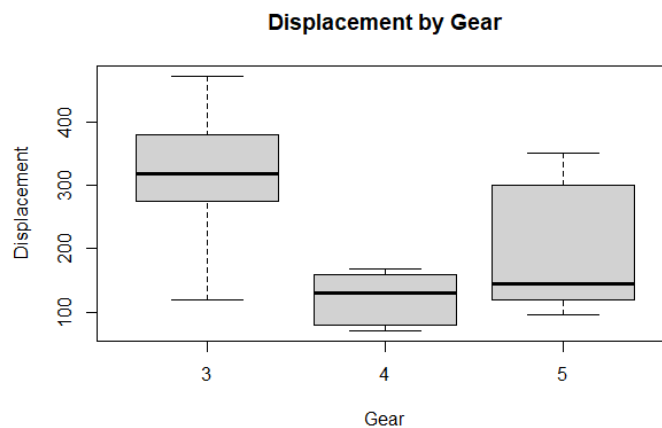
Syntax: `boxplot(x, data, notch, varwidth, names, main)`

- **x:** This parameter sets as a vector or a formula.
- **data:** This parameter sets the data frame.
- **notch:** This parameter is the label for horizontal axis.
- **varwidth:** This parameter is a logical value. Set as true to draw width of the box proportionate to the sample size.
- **main:** This parameter is the title of the chart.
- **names:** This parameter are the group labels that will be showed under each boxplot.

Ex:

```
data(mtcars)
```

```
boxplot(displacement ~ gear, data = mtcars, main = "Displacement by Gear", xlab = "Gear", ylab = "Displacement")
```

**(3)Histogram:**

A histogram contains a rectangular area to display the statistical information which is proportional to the frequency of a variable and its width in successive numerical intervals.

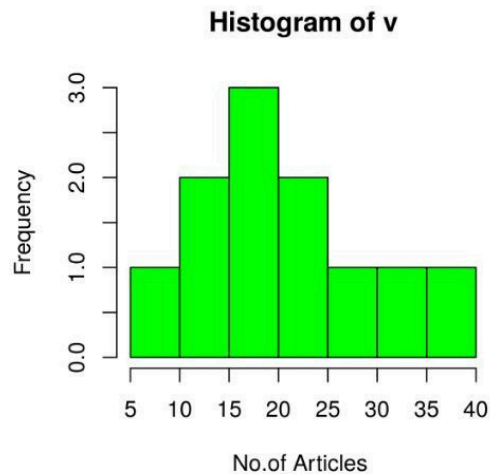
Syntax: `hist(v, main, xlab, xlim, ylim, breaks, col, border)`

- **v:** This parameter contains numerical values used in histogram.
- **main:** This parameter main is the title of the chart.
- **col:** This parameter is used to set color of the bars.
- **xlab:** This parameter is the label for horizontal axis.
- **border:** This parameter is used to set border color of each bar.
- **xlim:** This parameter is used for plotting values of x-axis.
- **ylim:** This parameter is used for plotting values of y-axis.
- **breaks:** This parameter is used as width of each bar.

Ex:

```
v <- c(19, 23, 11, 5, 16, 21, 32, 14, 19, 27, 39)
```

```
hist(v, xlab = "No.of Articles ",col = "green", border = "black")
```



(4) Scatter Plots:

A scatter plot is a set of dotted points representing individual data pieces on the horizontal and vertical axis. In a graph in which the values of two variables are plotted along the X-axis and Y-axis, the pattern of the resulting points reveals a correlation between them.

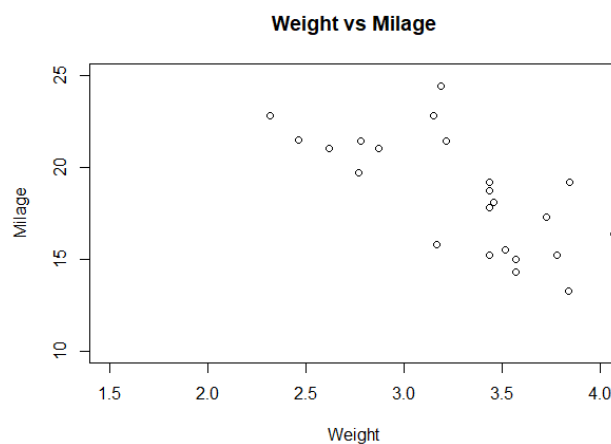
Syntax: `plot(x, y, main, xlab, ylab, xlim, ylim, axes)`

- **x:** This parameter sets the horizontal coordinates.
- **y:** This parameter sets the vertical coordinates.
- **xlab:** This parameter is the label for horizontal axis.
- **ylab:** This parameter is the label for vertical axis.
- **main:** This parameter main is the title of the chart.
- **xlim:** This parameter is used for plotting values of x.
- **ylim:** This parameter is used for plotting values of y.
- **axes:** This parameter indicates whether both axes should be drawn on the plot.

Ex:

```
input <- mtcars[, c('wt', 'mpg')]
```

```
plot(x = input$wt, y = input$mpg, xlab = "Weight", ylab = "Milage", xlim = c(1.5, 4), ylim = c(10, 25), main = "Weight vs Milage")
```



(5) Pie Chart:

A pie chart is a circular statistical graphic, which is divided into slices to illustrate numerical proportions. It depicts a special chart that uses "pie slices", where each sector shows the relative sizes of data.

Syntax: `pie(x, labels, radius, main, col, clockwise)`

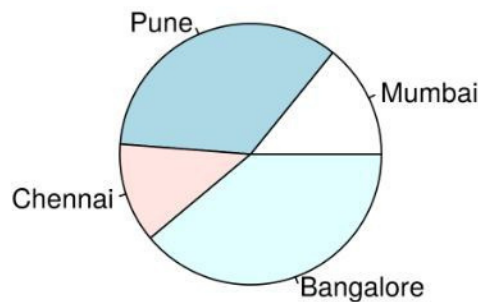
- **x:** This parameter is a vector that contains the numeric values which are used in the pie chart.
- **labels:** This parameter gives the description to the slices in pie chart.
- **radius:** This parameter is used to indicate the radius of the circle of the pie chart.(value between -1 and +1).
- **main:** This parameter is represents title of the pie chart.
- **clockwise:** This parameter contains the logical value which indicates whether the slices are drawn clockwise or in anti clockwise direction.
- **col:** This parameter give colors to the pie in the graph.

Ex:

```
geeks<- c(23, 56, 20, 63)
```

```
labels <- c("Mumbai", "Pune", "Chennai", "Bangalore")
```

pie(geeks, labels)

**(5) Line Graphs:**

A line graph is a chart that is used to display information in the form of a series of data points. It utilizes points and lines to represent change over time.

Syntax: `plot(v, type, col, xlab, ylab)`

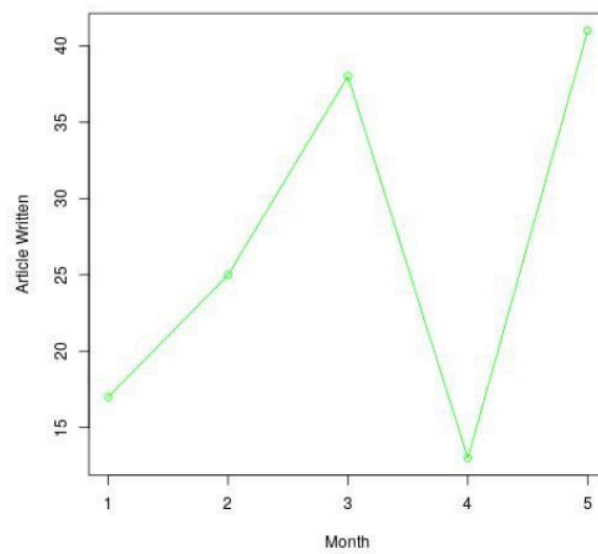
- **v:** This parameter is a contains only the numeric values
- **type:** This parameter has the following value:
 - "p" : This value is used to draw only the points.
 - "l" : This value is used to draw only the lines.
 - "o" : This value is used to draw both points and lines
- **xlab:** This parameter is the label for x axis in the chart.
- **ylab:** This parameter is the label for y axis in the chart.
- **main:** This parameter main is the title of the chart.
- **col:** This parameter is used to give colors to both the points and lines.

Ex:

```
v <- c(17, 25, 38, 13, 41)
```

```
plot(v, type = "o", col = "green", xlab = "Month", ylab = "Article Written", main = "Article Written chart")
```

Article Written chart



IMPORTANCE OF VISUALIZATIONS

Principles of Communicating Data

1. **Clarity and Simplicity** : Visualizations should convey information clearly and concisely. Avoid clutter and unnecessary elements that can confuse the audience.
2. **Relevance** : Include only data that is relevant to the message you want to convey. Extraneous information can dilute the impact of your visualization.
3. **Accuracy**: Ensure that your visual representation accurately reflects the underlying data. Misleading visualizations can lead to misinterpretation and incorrect conclusions.
4. **Consistency**: Use consistent scales, units, and formatting throughout your visualizations to maintain coherence and facilitate comparison.
5. **Audience Awareness**: Tailor your visualizations to your audience's level of understanding and familiarity with the data. Use appropriate terminology and avoid jargon that may alienate or confuse viewers.

Principles of Encoding Data to Make Visualizations

1. **Visual Encodings**: Choose appropriate visual elements (like bars, lines, points) to represent different types of data (quantitative, categorical, temporal).
2. **Use of Gestalt Principles**: Leverage principles like proximity, similarity, and continuity to organize and group data effectively within your visualizations.
3. **Hierarchy**: Employ visual hierarchy to emphasize important data points or trends. This can be achieved through variations in size, color, or position.
4. **Interactivity**: Consider incorporating interactive elements (such as tooltips, filters, or zoom) to allow users to explore the data more deeply and dynamically.

Importance of Color in Visualizations

1. **Differentiation**: Color helps differentiate between categories or data groups, making it easier for viewers to discern patterns and relationships.
2. **Emphasis**: Use color to highlight important data points or trends within your visualizations. This draws attention and guides viewers towards key insights.
3. **Accessibility**: Ensure that your color choices are accessible to all viewers, including those with color vision deficiencies. Use color combinations that have sufficient contrast and consider providing alternative visual cues (like patterns or shapes) in addition to color.
4. **Consistency**: Maintain consistency in your color scheme across related visualizations or within a single visualization. Consistent use of color enhances comprehension and reduces cognitive load.
5. **Contextual Meaning**: Be mindful of cultural or contextual associations with colors. Colors can carry symbolic meanings that may influence interpretation, so choose colors that align with the message or tone you wish to convey.

Plots Using Basic R

R is a powerful tool for statistical analysis and data visualization. It offers a wide range of functions for creating various types of plots. Here, we'll explore how to create some common plots using basic R plotting functions.

1. Scatter Plot

A scatter plot displays values for typically two variables for a set of data. It's useful for visualizing the relationship between two continuous variables.

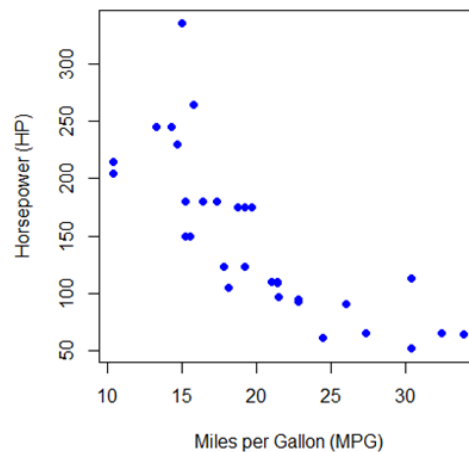
Load the mtcars dataset

```
data <- mtcars
```

Simple scatter plot of mpg vs hp

```
plot(data$mpg, data$hp,
     main="Scatter Plot of MPG vs Horsepower",
     xlab="Miles per Gallon (MPG)",
     ylab="Horsepower (HP)",
     pch=19, # Solid circle
     col="blue")
```

Scatter Plot of MPG vs Horsepower



2. Line Plot

Line plots are used to display data points connected by straight lines. This is often used to visualize data trends over time.

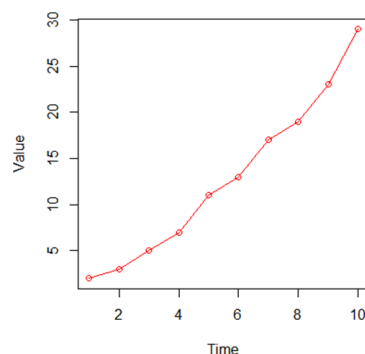
Sample data for a line plot

```
time <- 1:10
value <- c(2, 3, 5, 7, 11, 13, 17, 19, 23, 29)
```

Line plot

```
plot(time, value,
     type="o", # Line and points
     main="Line Plot",
     xlab="Time",
     ylab="Value",
     col="red")
```

Line Plot



3. Bar Plot

Bar plots are used to display categorical data with rectangular bars representing the frequency or proportion of each category.

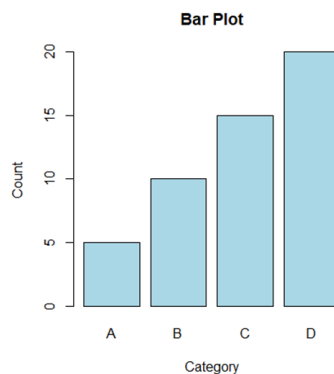
Sample data for a bar plot

```
categories <- c("A", "B", "C", "D")
```

```
counts <- c(5, 10, 15, 20)
```

Bar plot

```
barplot(counts,  
names.arg=categories,  
main="Bar Plot",  
xlab="Category",  
ylab="Count",  
col="lightblue")
```

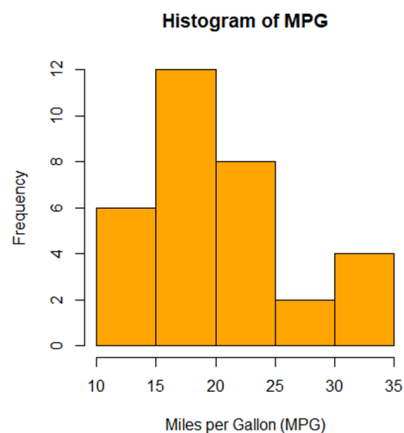


4. Histogram

Histograms are used to represent the distribution of a continuous variable by dividing the data into bins and plotting the frequency of data points in each bin.

Histogram of mpg from the mtcars dataset

```
hist(data$mpg,  
main="Histogram of MPG",  
xlab="Miles per Gallon (MPG)",  
ylab="Frequency",  
col="orange",  
border="black")
```

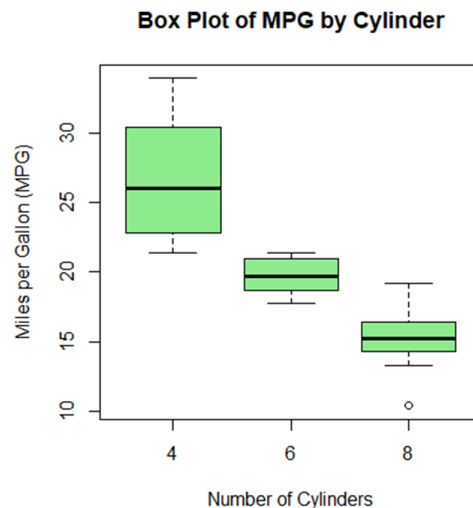


5. Box Plot

Box plots are used to summarize the distribution of a continuous variable and highlight the median, quartiles, and potential outliers.

Box plot of mpg by number of cylinders

```
data$cyl <- as.factor(data$cyl)
boxplot(mpg ~ cyl, data=data,
main="Box Plot of MPG by Cylinder",
xlab="Number of Cylinders",
ylab="Miles per Gallon (MPG)",
col="lightgreen")
```

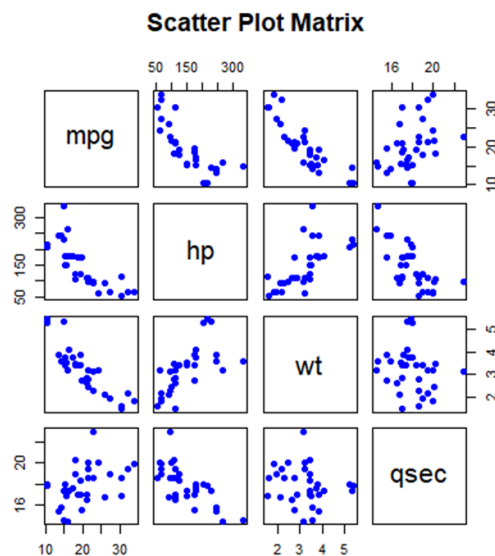


6. Scatter Plot Matrix

A scatter plot matrix is useful for visualizing relationships between multiple pairs of variables.

Scatter plot matrix for selected variables

```
pairs(~ mpg + hp + wt + qsec, data=data,
main="Scatter Plot Matrix",
pch=19, # Solid circle
col="blue")
```



Week-3.1 Plots With One Categorical and One Continuous Variable

R provides various functions to create plots for visualizing data with one categorical and one continuous variable. These plots help in understanding the distribution of a continuous variable across different categories. Below, we'll discuss different types of plots and the functions in R used to create them.

Functions in R for Plotting

plot(): Generic function for plotting.

barplot(): Creates bar plots.

hist(): Creates histograms.

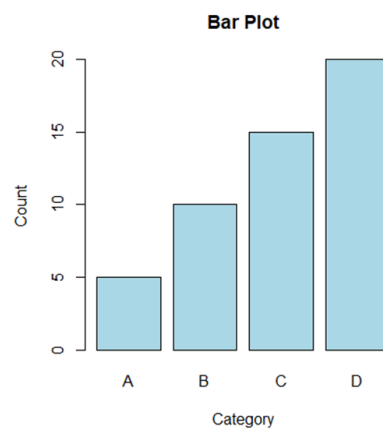
boxplot(): Creates box plots.

ggplot(): Part of the ggplot2 package, allows for more advanced and customizable plotting.

Plots with One Categorical Variable

1. Bar Plot

A bar plot is useful for visualizing the frequency or proportion of different categories.



2. Pie Chart

A pie chart represents the proportion of different categories.

```
categories <- c("A", "B", "C", "D")
```

```
counts <- c(5, 10, 15, 20)
```

Pie chart

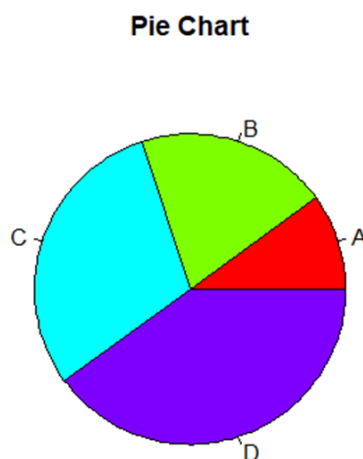
```
pie(counts,
```

```
  labels=categories,
```

```
  main="Pie Chart",
```

```
  col=rainbow(length(categories)))
```

presents the proportion of different categories as slices of a pie.



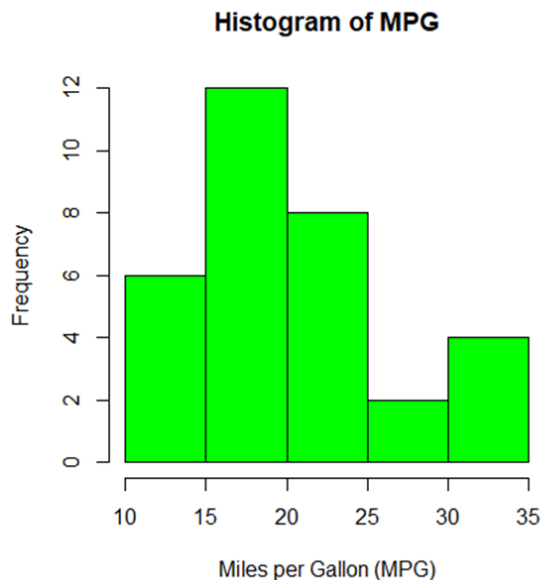
Plots with One Continuous Variable

1. Histogram

A histogram shows the distribution of a continuous variable by dividing the data into bins.

Histogram of mpg from the mtcars dataset

```
data <- mtcars
hist(data$mpg,
     main="Histogram of MPG",
     xlab="Miles per Gallon (MPG)",
     ylab="Frequency",
     col="green",
     border="black")
```



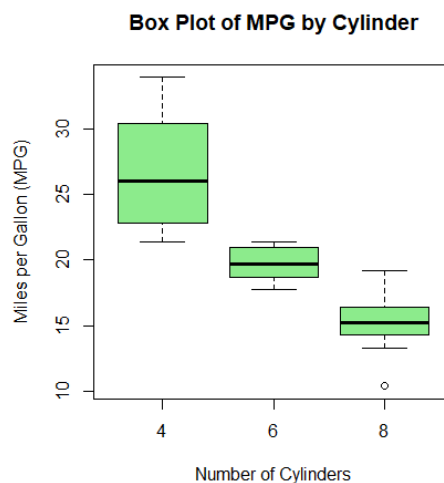
Plots with One Categorical and One Continuous Variable

1. Box Plot

A box plot summarizes the distribution of a continuous variable across different categories, highlighting the median, quartiles, and potential outliers.

Box plot of mpg by number of cylinders

```
data$cyl <- as.factor(data$cyl)
boxplot(mpg ~ cyl, data=data,
     main="Box Plot of MPG by Cylinder",
     xlab="Number of Cylinders",
     ylab="Miles per Gallon (MPG)",
     col="lightgreen")
```



WEEK-3.2

Plots with Two Continuous Variables

Visualizing the relationship between two continuous variables can be achieved through various types of plots, primarily scatter plots. We can control various aesthetics such as color, size, and shape to add additional dimensions to the visualization.

1) Scatterplot

Scatter plot of mpg vs hp

```
plot(data$mpg, data$hp,
      main = "Scatter Plot of MPG vs Horsepower",
      xlab = "Miles per Gallon (MPG)",
      ylab = "Horsepower (HP)",
      col = "blue", # Color of points
      pch = 19,     # Point shape (solid circle)
      cex = 1.5)    # Point size (1.5 times default)
```

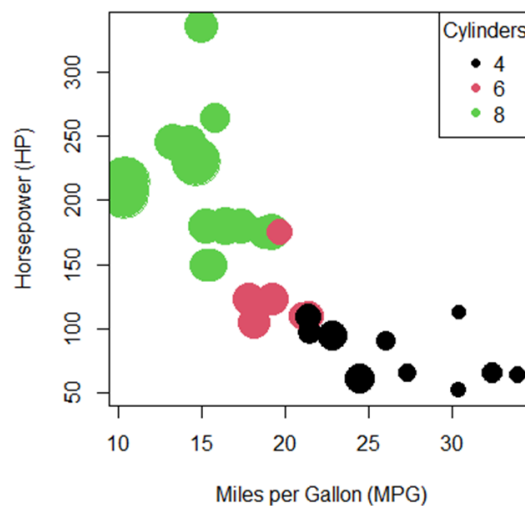
Scatter plot with color and size based on another variable

```
plot(data$mpg, data$hp,
      main = "Scatter Plot of MPG vs Horsepower",
      xlab = "Miles per Gallon (MPG)",
      ylab = "Horsepower (HP)",
      col = as.factor(data$cyl), # Color by number of cylinders
      pch = 19,
      cex = data$wt)            # Size by weight
```

Add a legend

```
legend("topright", legend = levels(as.factor(data$cyl)), col = 1:3, pch = 19, title = "Cylinders")
```

Scatter Plot of MPG vs Horsepower



EXERCISE 4:GGPLOT2 IN R

ggplot2 is a powerful data visualization package in R that allows users to create sophisticated and customizable plots with ease. Developed by Hadley Wickham, **ggplot2** follows the Grammar of Graphics concept, which defines a set of rules for building plots.

Here's a brief overview of **ggplot2**:

1. Layered Approach: **ggplot2** constructs plots in layers, where each layer represents a different aspect of the plot (data, aesthetics, geometric objects, statistical transformations, etc.). This layered approach allows for easy modification and customization of plots.

2. Grammar of Graphics: **ggplot2** follows the Grammar of Graphics, which breaks down the process of creating a plot into components such as data, aesthetics (visual properties like color, size, shape), and geometric objects (points, lines, bars). This makes it intuitive to create complex plots by combining simple components.

3. Data Mapping: **ggplot2** maps data variables to aesthetic properties of the plot (e.g., mapping a variable to the color or size of points). This allows for easy visualization of patterns and relationships in the data.

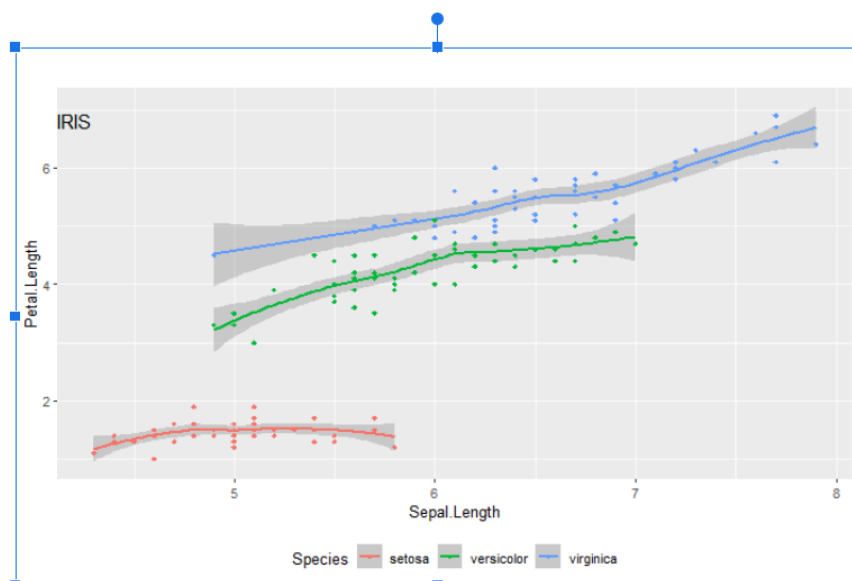
4. Flexible and Customizable: **ggplot2** provides extensive options for customizing plots, including changing colors, adding annotations, adjusting scales, and more. Users can easily tweak the appearance of plots to suit their needs.

5. Support for Faceting: **ggplot2** supports faceting, allowing users to create multiple plots (facets) based on subsets of the data. This is useful for exploring relationships within different groups of data.

Overall, **ggplot2** is widely used for creating publication-quality graphics in R due to its flexibility, ease of use, and extensive customization options. It has become a standard tool for data visualization and exploration in the R programming language.

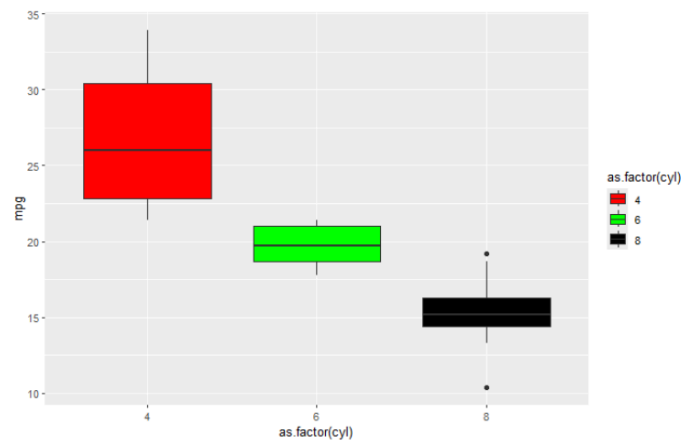
Code:

```
install.packages("ggplot2")
library(ggplot2)
data(iris)
dim(iris)
head(iris)
summary(iris)
ggplot(iris,aes(Sepal.Length,Petal.Length,col=Species))+geom_smooth()+geom
_point( shape
=18)+theme(legend.position="bottom")+ggtitle("IRIS")+theme(plot.title=element
_text(margin=margin(t=40,b=-30)))
```

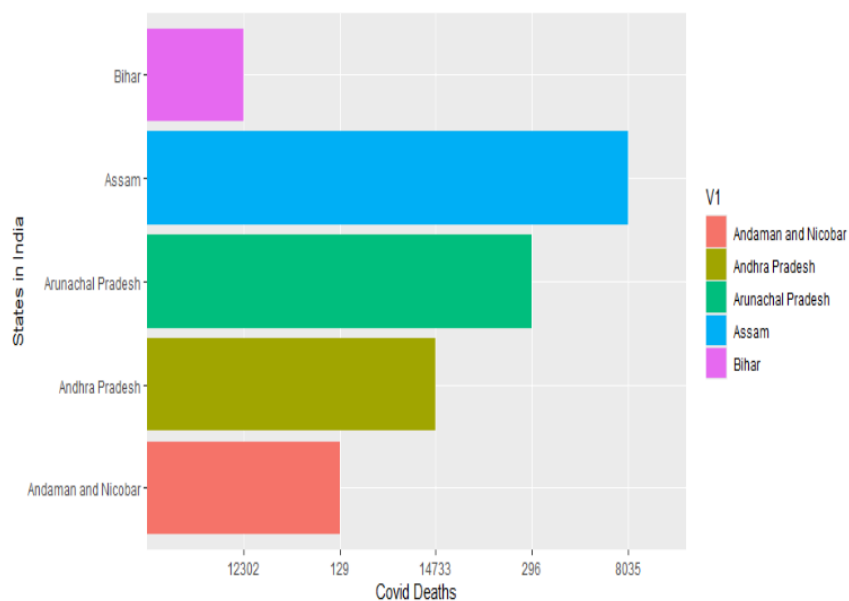
Output:

Code:

```
install.packages('tidyverse')
library(tidyverse)
data(mtcars)
dim(mtcars)
summary(mtcars)
glimpse(mtcars)
head(mtcars)
ggplot(mtcars,aes(x=as.factor(cyl),y=mpg,fill=as.factor(cyl)))+geom_boxplot()+scale_fill_manual(values=c('red','green','black'))
```

Output:**Code:**

```
data(covid)
summary(covid)
ggplot(covid[c(2:6),],aes(x=V1,y=V5,fill=V1))+geom_bar(stat='identity')+
coord_flip()+labs(y="Covid Deaths",x="States in India")
```

Output:

CONTROLLING AESTHETICS LIKE COLOUR, SIZE, LEGEND AND FACETS.

Load the ggplot2 package

```
library(ggplot2)
```

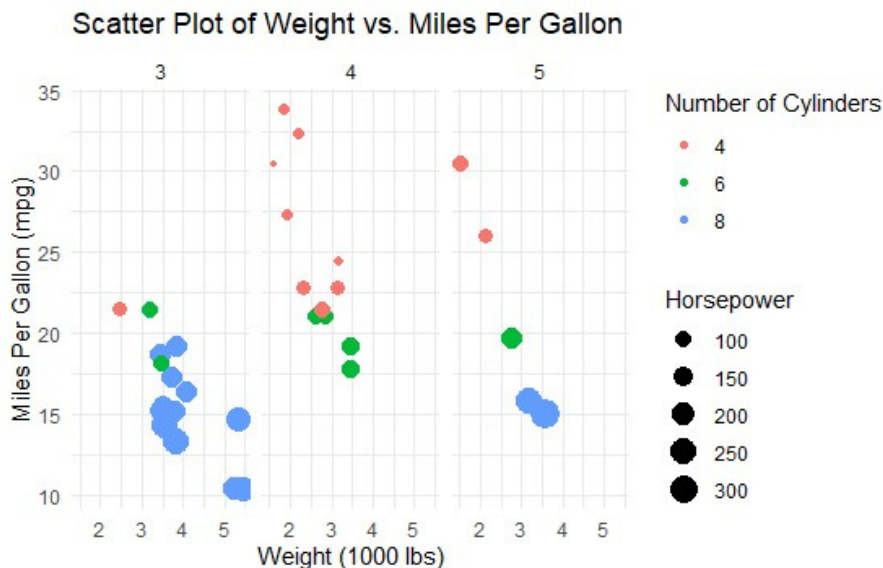
Load the mtcars dataset

```
data("mtcars")
```

Create a ggplot with custom aesthetics

```
ggplot(data = mtcars, aes(x = wt, y = mpg)) +  
  geom_point(aes(color = factor(cyl), size = hp)) + # Map color to the number of cylinders and size to  
  horsepower  
  labs(title = "Scatter Plot of Weight vs. Miles Per Gallon",  
        x = "Weight (1000 lbs)",  
        y = "Miles Per Gallon (mpg)",  
        color = "Number of Cylinders",  
        size = "Horsepower") +  
  theme_minimal() + # Use a minimal theme  
  facet_wrap(~ gear) # Facet by the number of gears
```

OUTPUT



1. Color

Categorical Coloring: Differentiate data points by a categorical variable using colors.

Example: Color data points by the number of cylinders (cyl).

Continuous Coloring: Apply a gradient color scale to a continuous variable.

Example: Color data points by horsepower (hp) with a gradient.

Custom Color Palette: Define and use a specific set of colors for different categories.

2. Size

Variable Point Sizes: Vary the size of data points based on a continuous variable.

Example: Change point size according to weight (wt).

Fixed Point Size: Set a fixed size for all data points for emphasis or clarity.

3. Legend

Legend Title and Labels: Customize the title and labels of the legend.

Example: Rename legend titles to make them more descriptive.

Legend Position: Change the position of the legend (e.g., top, bottom, left, right).

Legend Appearance: Adjust the appearance such as background, spacing, and text size.

4. Facets

Faceting by One Variable: Create multiple plots based on one categorical variable.

Example: Create separate plots for each number of cylinders (cyl).

Faceting by Two Variables: Create a grid of plots based on two categorical variables.

Example: Facet plots by both the number of cylinders (cyl) and the transmission type (am).

Combined Usage

Combining Aesthetics: Use multiple aesthetics (color, size, and facets) together for a richer visualization.

Example: Color by cyl, size by wt, and facet by gear.

By controlling these aesthetics, you can create more informative and visually appealing plots with ggplot2 to better convey the insights from your data.

WEEK-6 Data visualization

1. Importing the Data:

Data Source: Identify the source of your data, whether it's a file (CSV, Excel, etc.), a database, or an API.

Data Format: Understand the structure of your data (rows and columns) and how its organized.

Data Cleaning: Preprocess your data to handle missing values, remove duplicates, and format data types appropriately.

Load Data: Import your data into a data analysis tool or programming environment such as Python(using libraries like Pandas), R, or Excel.

2. Dimensions and Measures:

Identify Variables:

Dimensions: Look for categorical variables that represent categories or groups. Examples include product categories, customer segments, or geographical regions.

Measures: Identify numerical variables that provide quantitative information. Examples include sales revenue, temperature, or number of units sold.

Role of Variables:

Dimensions: Typically used for grouping or categorizing data. They are often represented on the x-axis, y-axis, or as labels in visualizations.

Measures: Provide the quantitative values that you want to visualize. They are often represented by the size, length, or position of visual elements such as bars in a barchart or points in a scatter plot.

3. Color Coding:

Types of Variables and Color Coding:

Categorical Variables: Assign distinct colors to different categories to differentiate them visually. Use a color palette with contrasting hues.

Sequential Variables: Use a gradient of color to represent the progression or intensity of a variable.

Lighter shades typically represent lower values, while darker shades represent higher values.

Diverging Variables: Use diverging color schemes when there is a midpoint or critical value. Assign different colors to values below and above the midpoint, with the midpoint represented by a neutralcolor.

Color Considerations:

Color Accessibility: Ensure that your color choices are accessible to all users, including those with color vision deficiencies. Use color-blind friendly palettes.

Consistency: Maintain consistency in color usage across different visualizations to aid interpretation.

Contrast: Use contrasting colors to highlight important information or data points.

Meaningful Encoding: Make sure that the color encoding is meaningful and aligned with the data being represented.

1. Importing Data

```
library(ggplot2)
data <- mtcars
```

Display the first few rows of the dataset

```
head(data)
```

Output:

```
mpg cyl disp hp dratwt qsec vs am gear carb
Mazda RX4 21.0 6 160 110 3.90 2.620 16.46 0 14 4
Mazda RX4 Wag 21.0 6 160 110 3.90 2.875 17.02 0 14 4
Datsun 710 22.8 4108 93 3.85 2.320 18.61 11 4 1
Hornet 4 Drive 21.4 6 258 110 3.08 3.215 19.44 1 03 1
Hornet Sportabout 18.7 8 360 175 3.15 3.440 17.02 00 3 2
Valiant 18.1 6 225 105 2.76 3.460 20.22 1 03 1
```

2. Identifying Dimensions and Measures

In data visualization, dimensions are typically categorical variables, while measures are numerical variables. We can identify them by inspecting the dataset.

Display the structure of the dataset

```
str(data)
\
```

Output:

```
'data.frame': 32 obs. of 11 variables:
 $ mpg : num 21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num 6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num 160 160 108 258 360 ...
 $ hp : num 110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt : num 2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num 16.5 17 18.6 19.4 17 ...
 $ vs : num 0 0 1 1 0 1 0 1 1 1 ...
 $ am : num 1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num 4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

3. Applying Color Coding for Various Types of Variables

a. Bar Plot for Categorical Data library(ggplot2)

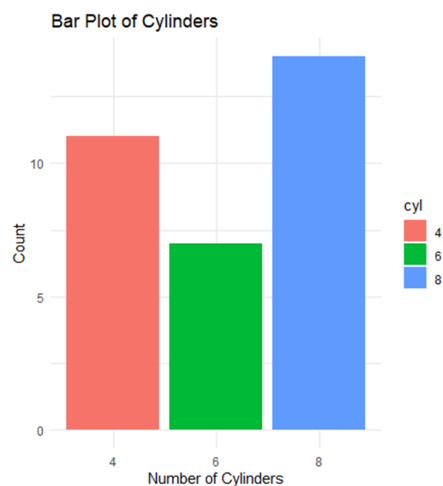
Bar plot

```
data$cyl <- as.factor(data$cyl)
```

```
ggplot(data, aes(x = cyl, fill = cyl)) + geom_bar() + labs(title = "Bar Plot of Cylinders", x = "Number of Cylinders", y = "Count") + theme_minimal()
```

Scatter plot

```
ggplot(data, aes(x = Sales, y = Profit, color = Category)) + geom_point() + labs(title = "Scatter Plot of Sales vs Profit", x = "Sales", y = "Profit") + theme_minimal()
```

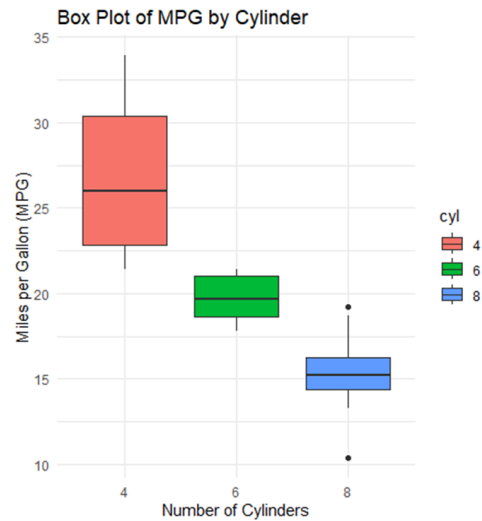
**b. Box Plot for Numerical Data across Categories****# Box plot****# Convert cyl to a factor (categorical variable)**

```
data$cyl <- as.factor(data$cyl)
```

Create a box plot of mpg by cyl

```
ggplot(data, aes(x = cyl, y = mpg, fill = cyl)) +  
geom_boxplot() +
```

```
labs(title = "Box Plot of MPG by Cylinder", x = "Number of Cylinders", y = "Miles per Gallon (MPG)") + theme_minimal()
```



4. Combining Dimensions and Measures in a Single Visualization

Faceted plot

```
data$cyl <- as.factor(data$cyl)
```

```
data$gear <- as.factor(data$gear)
```

Create a faceted plot of mpg vs hp by gear and cyl

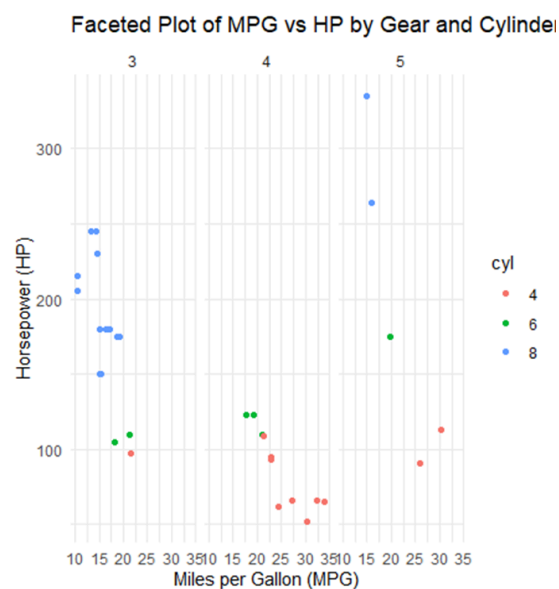
```
ggplot(data, aes(x = mpg, y = hp, color = cyl)) +
```

```
geom_point() +
```

```
facet_wrap(~ gear) +
```

```
labs(title = "Faceted Plot of MPG vs HP by Gear and Cylinders", x = "Miles per Gallon (MPG)", y = "Horsepower (HP)") +
```

```
theme_minimal()
```



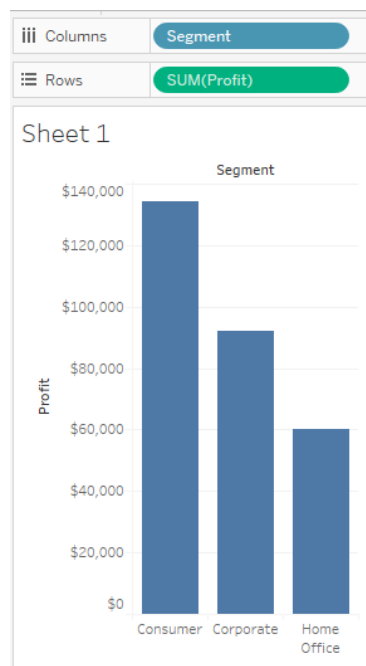
Exercise-7 Working with sheets

Columns and Rows shelves:

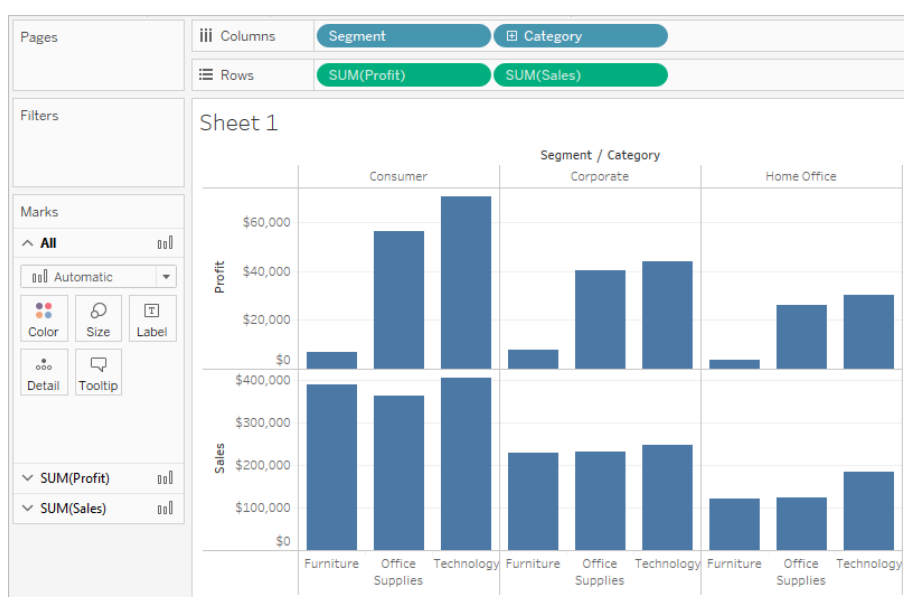
Drag fields from the Data pane to create the structure for your visualizations.

The **Columns** shelf creates the columns of a table, while the **Rows** shelf creates the rows of a table. You can place any number of fields on these shelves.

When you place a dimension on the **Rows** or **Columns** shelves, headers for the members of that dimension are created. When you place a measure on the **Rows** or **Columns** shelf, quantitative axes for that measure are created. As you add more fields to the view, additional headers and axes are included in the table and you get an increasingly detailed picture of your data.



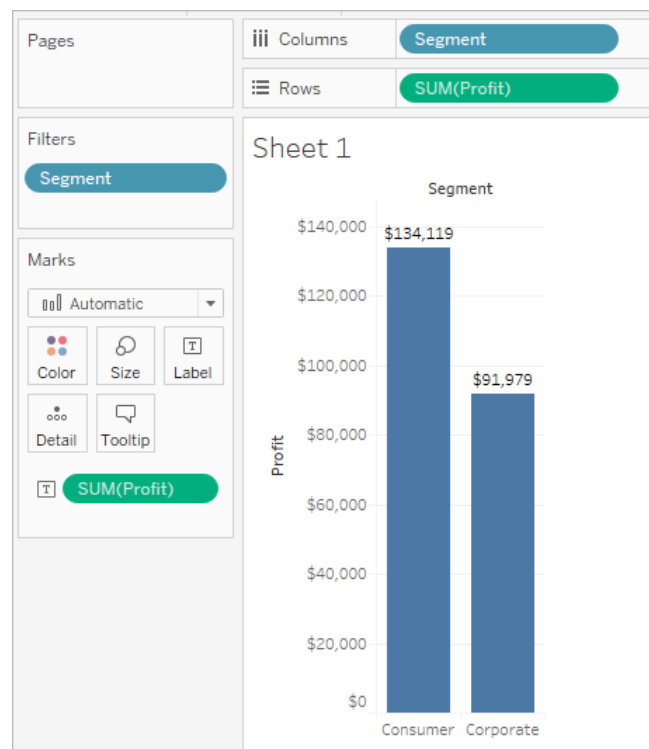
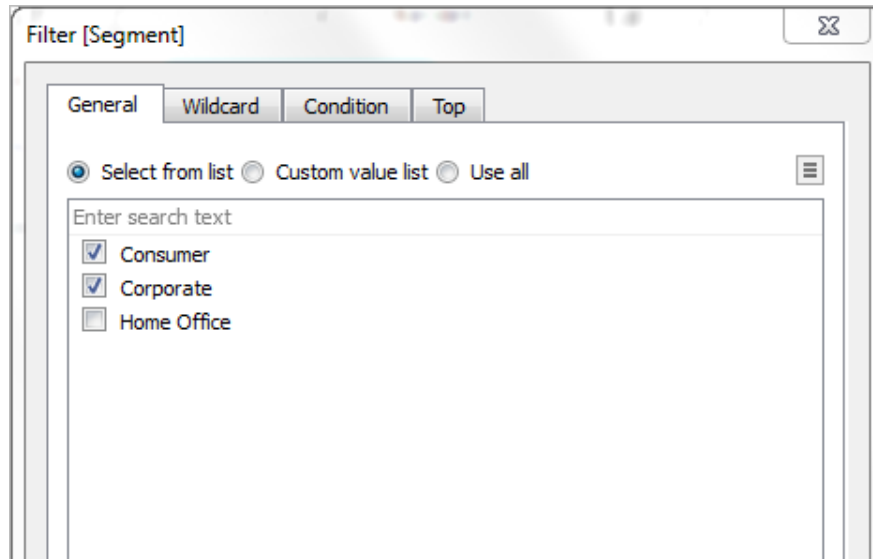
Adding more fields to the Rows and Columns shelves adds more rows, columns, and panes to the table.



Filters shelf:

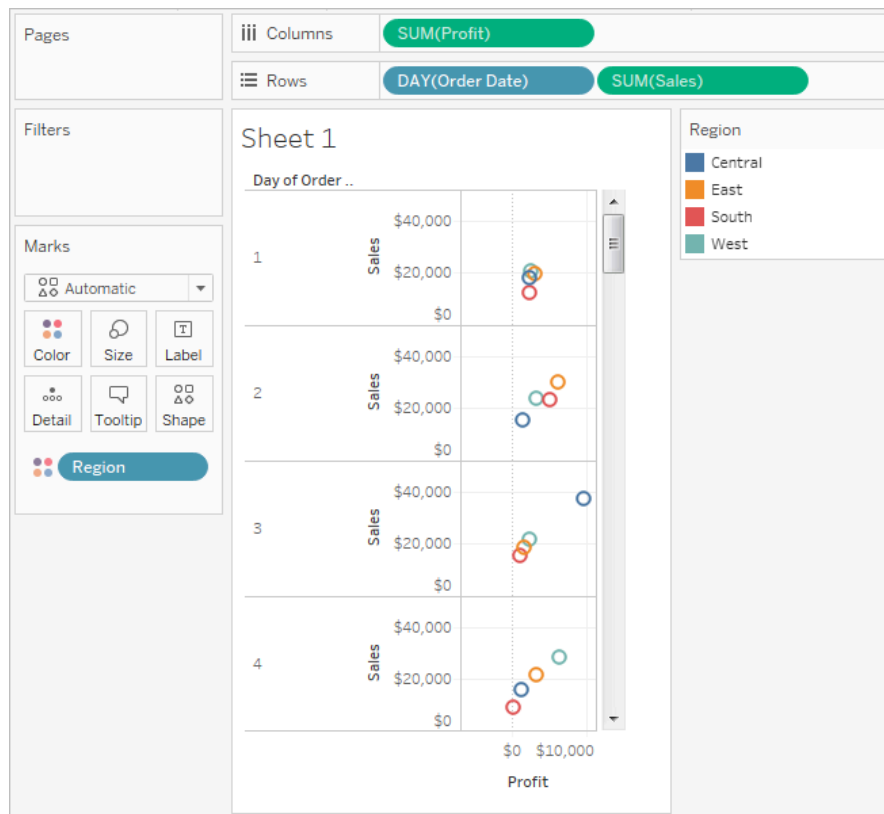
The Filters shelf allows you to specify which data to include and exclude. For example, you might want to analyze the profit for each customer segment, but only for certain shipping containers and delivery times. By placing fields on the Filters shelf, you can create such a view.

You can filter data using measures, dimensions, or both at the same time. Additionally, you can filter data based on the fields that make up the columns and rows of the table. This is called an internal filter. You can also filter data using fields that don't contribute headers or axes to the table. This is called an external filter. All filtered fields display on the Filters shelf



Pages shelf

The **Pages** shelf lets you break a view into a series of pages so you can better analyze how a specific field affects the rest of the data in a view. When you place a dimension on the **Pages** shelf you are adding a new row for each member in the dimension. When you place a measure on the **Pages** shelf, Tableau automatically converts the measure into a discrete measure.



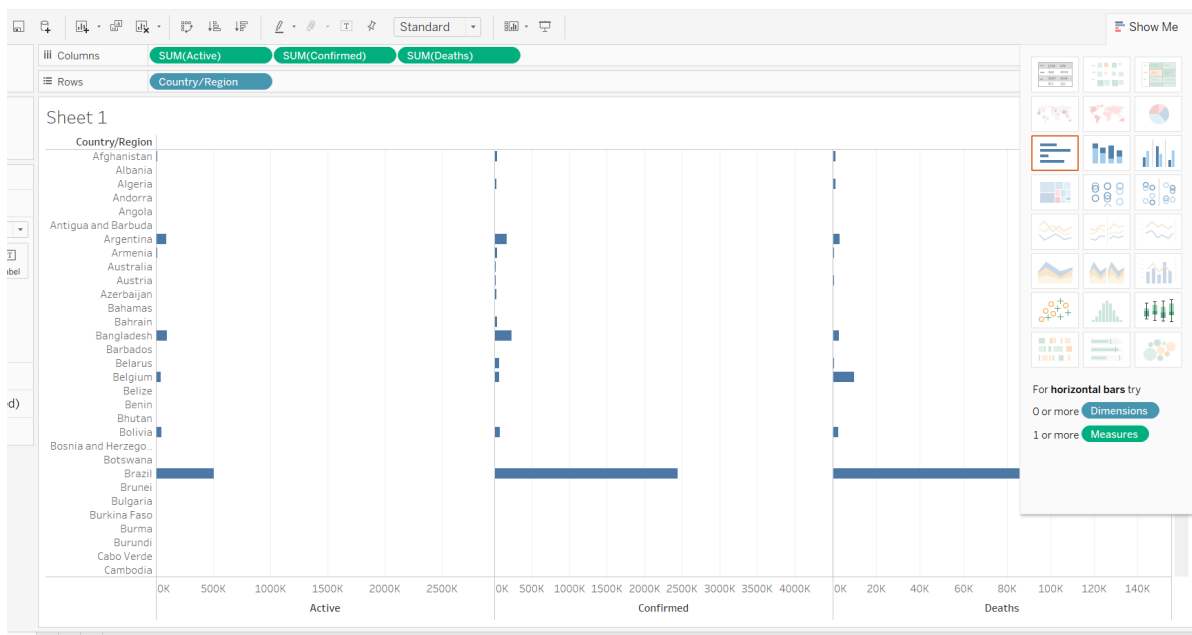
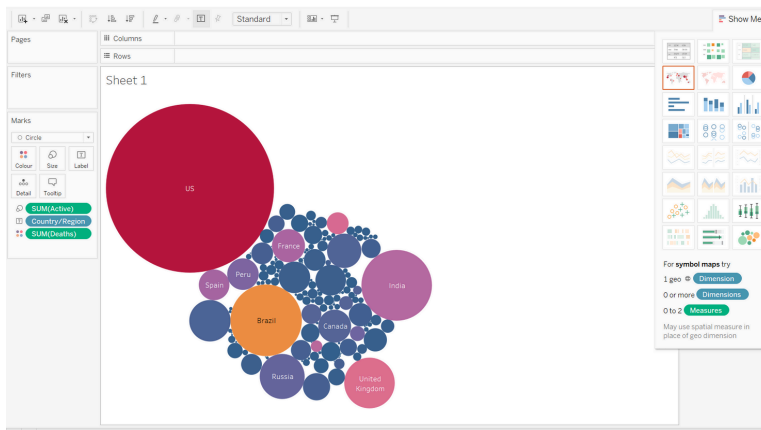
Show Me Card

Show Me creates a view based on the fields already used in the view and any fields you've selected in the Data pane. Open Show Me by clicking Show Me on the toolbar. Tableau automatically evaluates the selected fields and gives you the option of several types of views that would be appropriate for those fields. Show Me also highlights the visualization type that best matches the data.

To Visualize using Show Me Chart:

- In the Data pane, select the fields that you want to analyze. Hold the Ctrl key to make multiple selections.
- Click Show Me on the toolbar and then select the type of view you want to create.

For example, here we took a covid dataset and performed the following visualization:



There are 24 different charts in 'Show Me' and each have their requirements such as:

For **text tables** try

1 or more **Dimensions**

1 or more **Measures**

For **stacked bars** try

1 or more **Dimensions**

1 or more **Measures**

For **symbol maps** try

1 geo  **Dimension**

0 or more **Dimensions**

0 to 2 **Measures**

May use spatial measure in place of geo dimension

For **side-by-side bars** try

1 or more **Dimensions**

1 or more **Measures**

Requires at least 3 fields

For **pie charts** try

1 or more **Dimensions**

1 or 2 **Measures**

For **heat maps** try

1 or more **Dimensions**

1 or 2 **Measures**

For **highlight tables** try

1 or more **Dimensions**

1 **Measure**

For **horizontal bars** try

0 or more **Dimensions**

1 or more **Measures**

In conclusion the Show Me feature is a helpful aid to better understand the visuals that are built in Tableau. the "Show Me" card in Tableau is an essential tool that enhances the user experience by providing quick, intelligent visualization suggestions. It is particularly beneficial for beginners and those looking to expedite the process of creating effective data visualizations. By promoting best practices and allowing for customization, the "Show Me" card helps users of all skill levels produce insightful and impactful visual analyses.

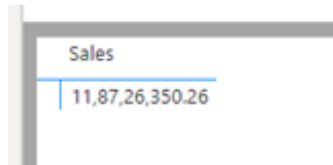
Calculations

Measures:

Measures in Power BI are a summarization of any data. So, it is important to have a summary of any data or representation of data. Although in Power BI, we have tools to create our measures based on the data itself. We also have the option to name the measures in the way we want.

Explicit Measures:

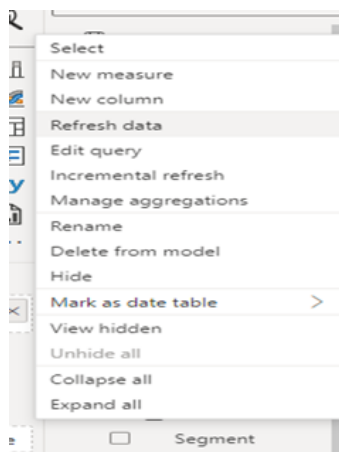
These are being created by entering the DAX function. These can be accessed anywhere in the report and can be used in other DAX calculations too.



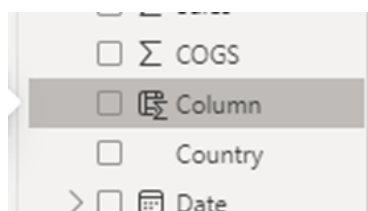
Creating Calculated columns:

In your Sales Report, you want to display product categories and subcategories as single values, like "Cell phones – Accessories", "Cell phones – Smartphones & PDAs", and so on. There's no field in the Fields list that gives you that data, but there's a ProductCategory field and a **ProductSubcategory** field, each in its own table. You can create a calculated column that combines values from these two columns. DAX formulas can use the full power of the model you already have, including relationships between different tables that already exist.

- To create your new column in the ProductSubcategory table, right-click or select the ellipsis ... next to ProductSubcategory in the Fields pane, and choose New column from the menu.

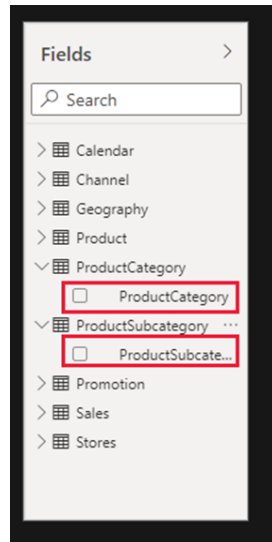


- By default, a new calculated column is named Column. If you don't rename it, new columns will be named Column 2, Column 3, and so on. You want your column to be more identifiable, so while the Column name is already highlighted in the formula bar, rename it by typing ProductFullCategory, and then type an equals(=) sign.

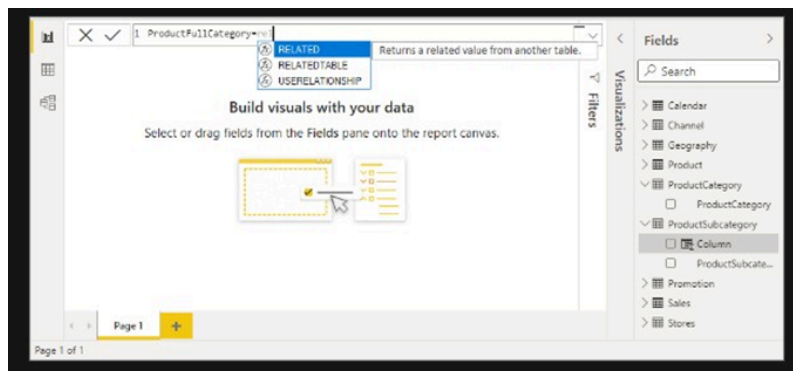


- You want the values in your new column to start with the name in the ProductCategory field. Because this column is in a different but related table, you can use the RELATED function to help you get it.

After the equals sign, type r. A dropdown suggestion list shows all of the DAX functions beginning with the letter R. Selecting each function shows a description of its effect. As you type, the suggestion list scales closer to the function you need. Select RELATED, and then press Enter.



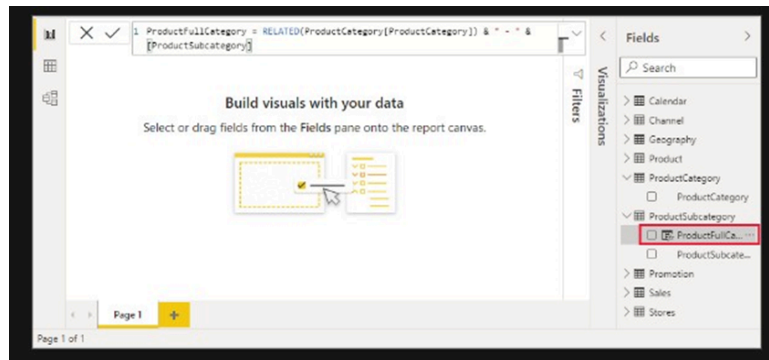
- You want the ProductCategory column from the ProductCategory table. Select ProductCategory[ProductCategory], press Enter, and then type a closing parenthesis.



- You want dashes and spaces to separate the ProductCategories and ProductSubcategories in the new values, so after the closing parenthesis of the first expression, type a space, ampersand (&), double-quote ("), space, dash (-), another space, another double-quote, and another ampersand. Your formula should now look like this:



- Enter an opening bracket ([), and then select the [ProductSubcategory] column to finish the formula.
- Complete the formula by pressing Enter or selecting the checkmark in the formula bar. The formula validates, and the ProductFullCategory column name appears in the ProductSubcategory table in the Fields pane.



Quick Table Calculations / Quick Measures:

Measures in Power BI help transform data as data often requires more complex calculations to answer some questions. Measures can be used for some of the most simplistic calculations such as sum, average, min and max values and many more. Let's start with an example to see how you can use quick measures to get to these insights.

Quick measure

Select a calculation to create a measure or describe the measure you need and we'll generate suggestions in DAX, which you can customize later.

Aggregate per category

- Average per category
- Variance per category
- Max per category
- Min per category
- Weighted average per category

Time intelligence

- Year-to-date total
- Quarter-to-date total
- Month-to-date total
- Year-over-year change
- Quarter-over-quarter change
- Month-over-month change
- Rolling average

Mathematical operations

- Addition
- Subtraction
- Multiplication
- Division
- Percentage difference
- Correlation coefficient
- Text
- Star rating
- Concatenated list of values

Filters

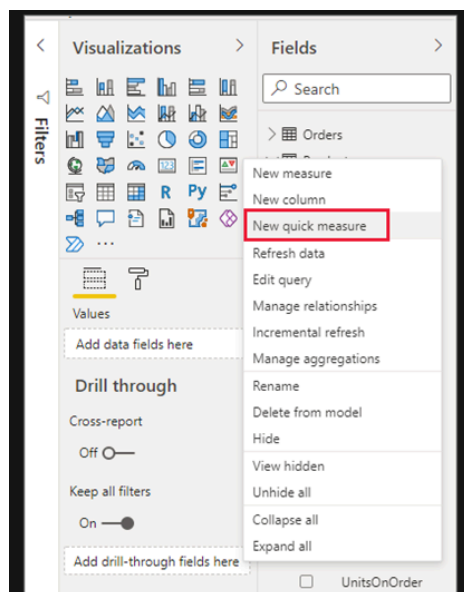
- Filtered value
- Difference from filtered value
- Percentage difference from filtered value
- Sales from new customers

Totals

- Running total
- Total for category (filters applied)
- Total for category (filters not applied)

Create a quick measure:

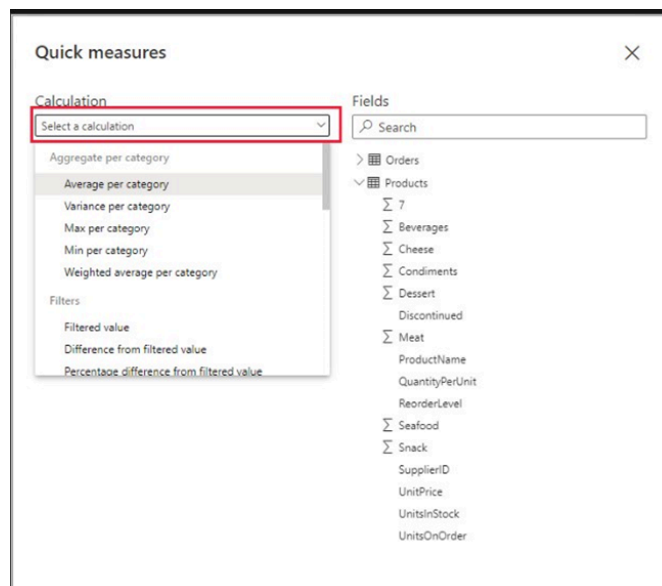
To create a quick measure in Power BI Desktop, right-click or select the ellipsis ... next to any item in the Fields pane, and choose New quick measure from the menu that appears.



You can also right-click or select the drop-down arrow next to any value in the Values well for an existing visual, and choose New quick measure from the menu.

When you select New quick measure, the Quick measures window appears, letting you choose the calculation you want and the fields to run the calculation against.

Choose the Select a calculation field to see a long list of available quick measures.



Parameters and Groups:

If you want to build a report where the user can choose what measure to show, you have two features available in Power BI: field parameters and calculation groups. There are pros and cons to either technique – however, we are not about to talk about those. We narrow our scenario down to a specific requirement: we want to change the color of the value depending on the measure selected.

For example, suppose we let users choose between Sales Amount, Margin, or Total Cost. In that case, we might provide visual feedback about the measure selected through different colors: black for Sales Amount, green for Margin, and red for Total Cost.

Using field parameters:

Field parameters use a calculated table tagged as a field parameter table. The slicer places a filter on the table, then Power BI detects the value selected to change the query and returns the value of the desired measure.

In our example, the user can choose between three possible metrics: Sales Amount, Total Cost, and Margin. Power BI created this calculated table:

```

1 Measure =
2 {
3   ( "Sales Amount", NAMEOF ( 'Sales'[Sales Amount] ), 0 ),
4   ( "Total Cost", NAMEOF ( 'Sales'[Total Cost] ), 1 ),
5   ( "Margin", NAMEOF ( 'Sales'[Margin] ), 2 )
6 }

```

The query that computes the result is generated dynamically. Power BI uses the slicer selection to generate a DAX query that computes the selected measure. The slicer selection is also part of the generated query, although this filter is unnecessary because the selected measure is already part of the query. However, it is handy if developers need to retrieve the name of the chosen measure.

Using calculation groups:

Field parameters are a simple and great solution for the scenario presented. However, they lack some flexibility and cannot be used for example in Excel. Developers can use calculation groups to obtain a behavior that is very similar to that of field parameters. The following calculation group creates the same effect as the field parameter:

```

1 -----
2 -- Calculation Group: 'Measure CG'
3 -----
4 CALCULATIONGROUP 'Measure CG'[Measure CG]
5
6     CALCULATIONITEM "Sales Amount" =
7         [Sales Amount],
8         Ordinal = 0
9
10    CALCULATIONITEM "Total Cost" =
11        [Total Cost],
12        Ordinal = 1
13
14    CALCULATIONITEM "Margin" =
15        [Margin],
16        Ordinal = 2

```


EXERCISE 9: Graphs for Analytics

Calculate Proportions and percentages, Comparing current to historical and Actual to Target :

Proportions: These are ratios or fractions that represent a part of a whole. In analytics, proportions could refer to how much of something (e.g., sales, expenses) relates to the total (e.g., total sales, total expenses).

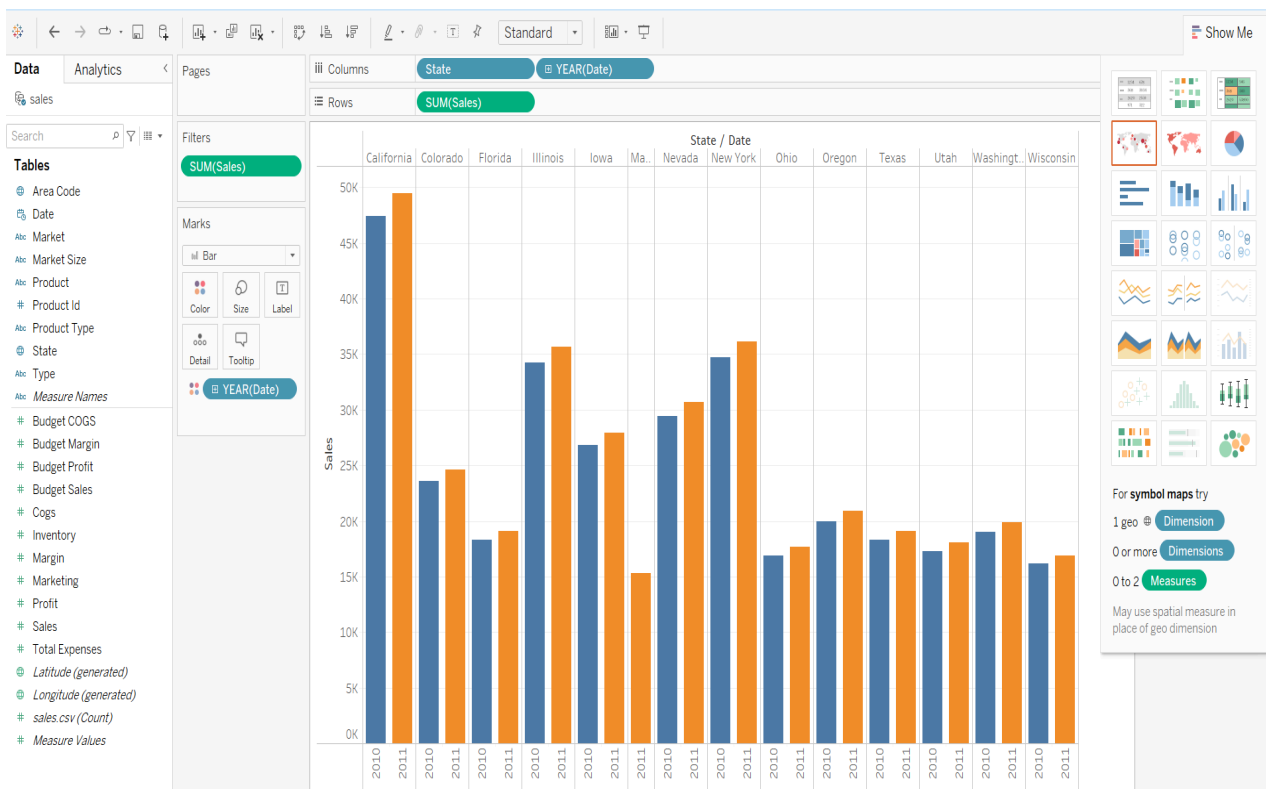
Percentages: Percentages are proportions expressed as parts per hundred. They are often used to compare different quantities or to show changes over time relative to a baseline.

Dataset : US STORES SALES

Proportions :

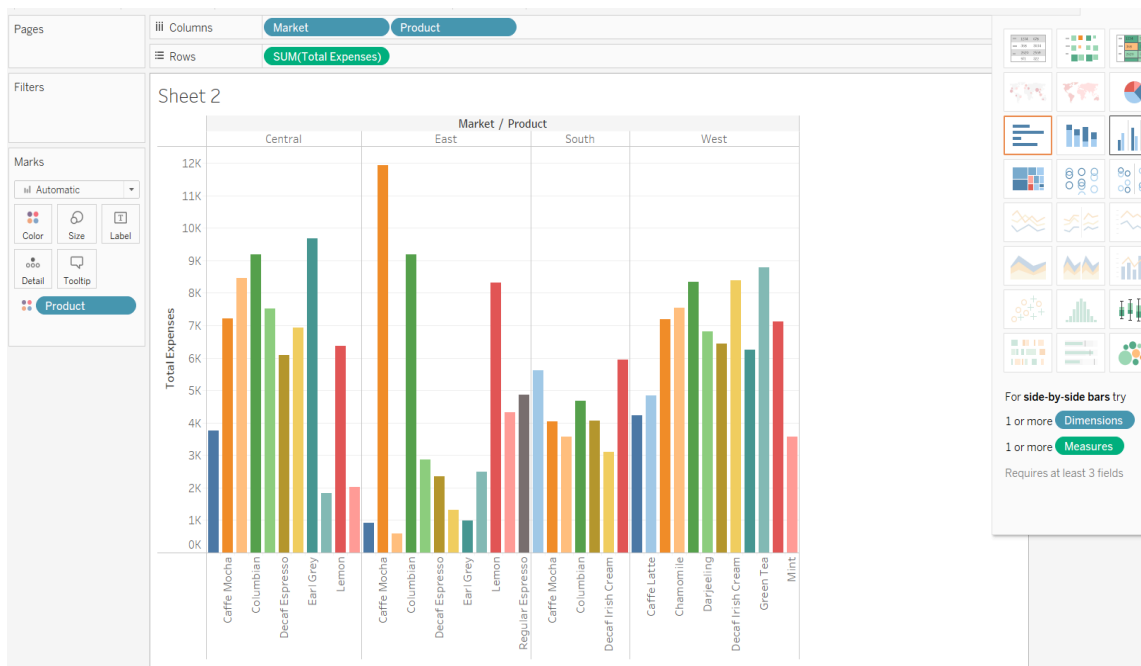
1. Comparing two consecutive years sales data of US for all the states ,taking States/dates in the column and sales as rows .

Bar Chart:



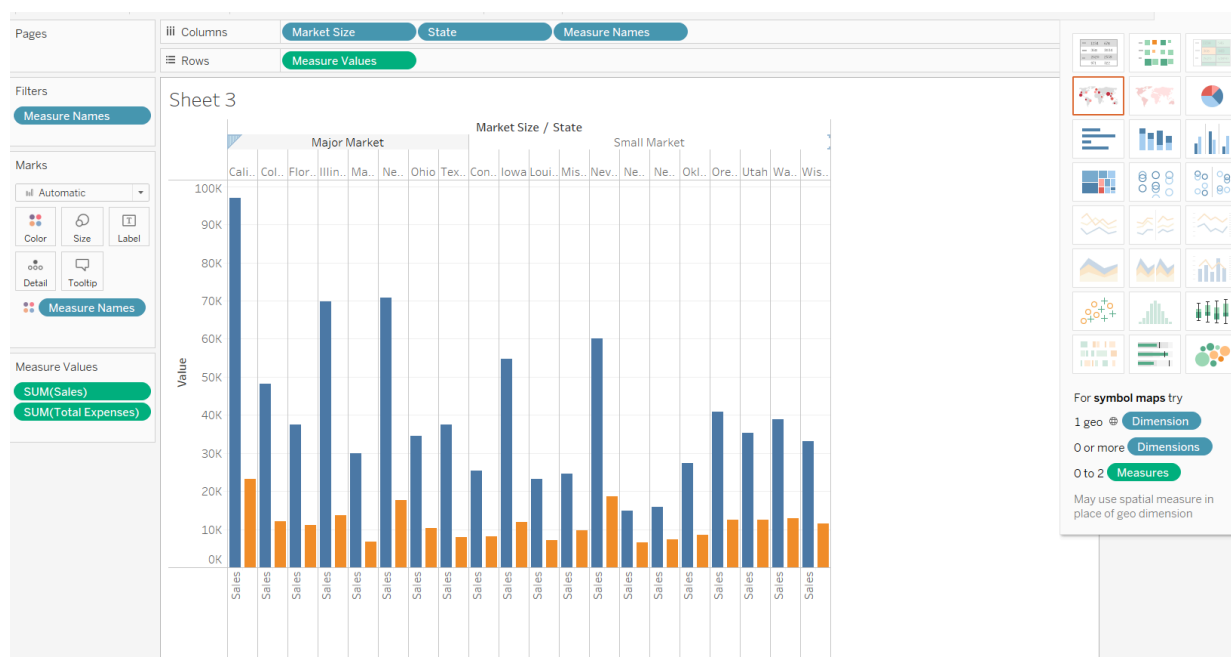
2. Comparing total expenses in regional areas in US with market and product ,taking market/product as columns and total expenses as rows .

Bar Chart:



3. Comparing the actual values (total expenses) and target values (sales) ,taking proportions using bar charts whereas total expenses/sales as columns and states/market size as row.

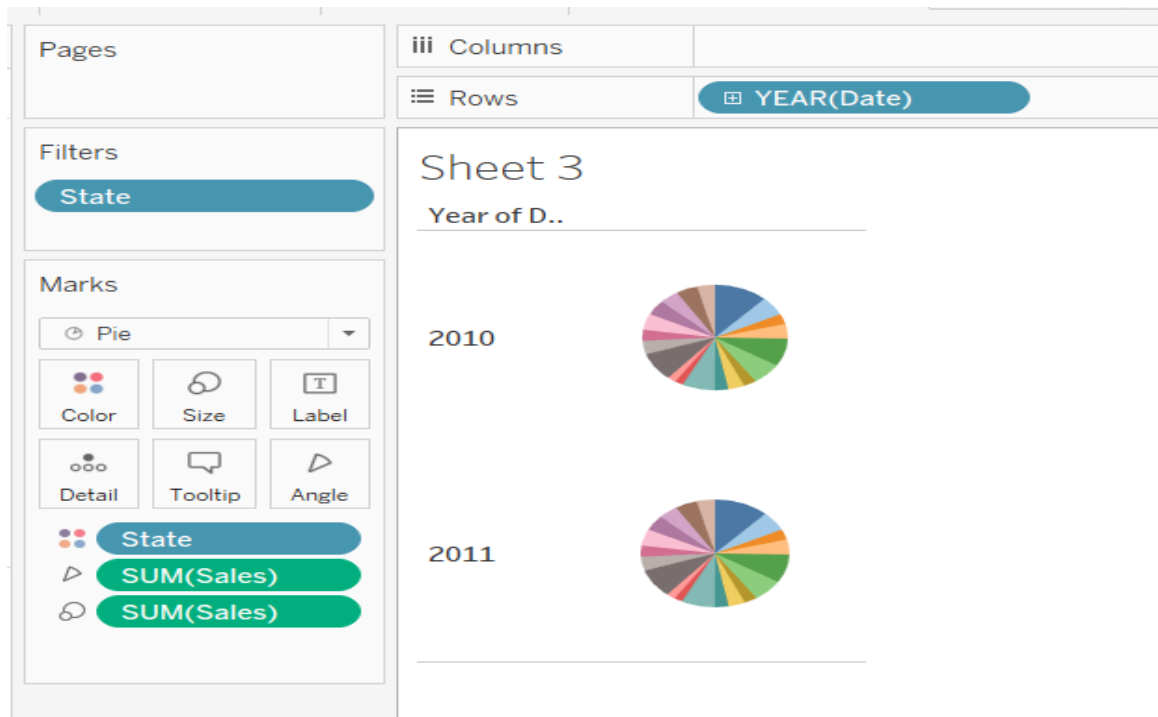
Bar Chart:



Percentages:

We use pie charts to show the percentages from the states ,markets, sales and profits of a region.

- 1.Comparing sales of region for two different years , a pie chart can give clear percentages about this comparison.

Pie Chart:

EXERCISE 10: Normal Distribution Variation

Mean Calculation:

- The mean is a measure of central tendency, representing the average value of a dataset. It is calculated by summing all the data points and then dividing by the number of points.
- `mean_mpg <- mean(mtcars$mpg)` computes the mean of the mpg column in the mtcars dataset.

$$\text{Mean } (\bar{x}) = \frac{\sum x}{n}$$

Median Calculation:

- The median is another measure of central tendency, representing the middle value in a dataset when the data points are sorted in ascending order. If there is an even number of observations, the median is the average of the two middle numbers.
- `median_mpg <- median(mtcars$mpg)` computes the median of the mpg column in the mtcars dataset.

Odd	Even
$\frac{n+1}{2}$	$\frac{n}{2}, \frac{n}{2} + 1$

Code :

```
data(mtcars)

mean_mpg <- mean(mtcars$mpg)
print(paste("Mean of mpg:", mean_mpg))

median_mpg <- median(mtcars$mpg)
print(paste("Median of mpg:", median_mpg))
```

Output :

Mean: 20.09
Median: 19.2

Plotting:

- Histogram: The histogram shows the frequency distribution of the mpg values. It is created using `geom_histogram()` with the y aesthetic set to density to normalize the counts.
- Normal Distribution Curve: This curve, added with `stat_function()`, represents the theoretical normal distribution based on the mean and standard deviation of the mpg data.
- Mean and Median Lines: Vertical dashed lines indicate the mean (`geom_vline(aes(xintercept = mean_mpg), ...)`) and median (`geom_vline(aes(xintercept = median_mpg), ...)`) values on the plot.
- Annotations: The `annotate()` function is used to add text labels for the mean and median on the plot.

Normal Distribution Variation and Uncertainty**Code :**

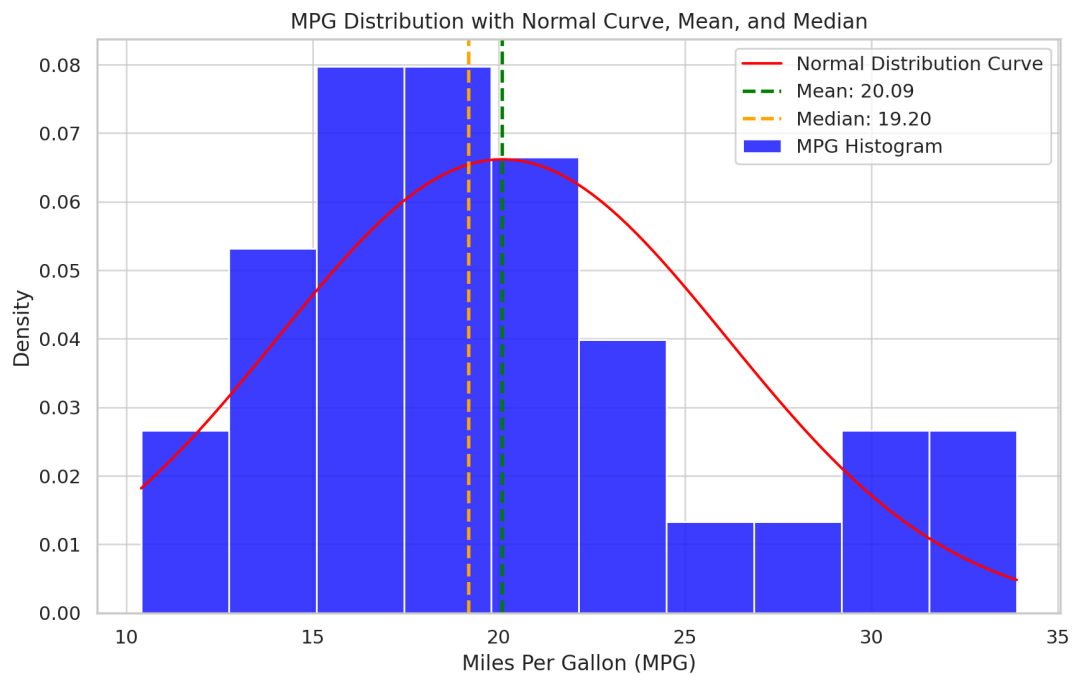
```
library(ggplot2)

data(mtcars)

mean_mpg <- mean(mtcars$mpg)
median_mpg <- median(mtcars$mpg)

ggplot(mtcars, aes(x = mpg)) +
  geom_histogram(aes(y = ..density..), bins = 10, fill = 'blue', color = 'black', alpha = 0.7) +
  stat_function(fun = dnorm, args = list(mean = mean_mpg, sd = sd(mtcars$mpg)), color = 'red', size = 1) +
  geom_vline(aes(xintercept = mean_mpg), color = 'green', linetype = "dashed", size = 1, show.legend =
TRUE) +
  geom_vline(aes(xintercept = median_mpg), color = 'orange', linetype = "dashed", size = 1, show.legend =
TRUE) +
  labs(title = "MPG Distribution with Normal Curve, Mean, and Median",
       x = "Miles Per Gallon (MPG)",
       y = "Density") +
  annotate("text", x = mean_mpg, y = 0.03, label = paste("Mean:", round(mean_mpg, 2)), color = "green",
vjust = -1.5) +
  annotate("text", x = median_mpg, y = 0.03, label = paste("Median:", round(median_mpg, 2)), color =
"orange", vjust = -1.5) +
  theme_minimal()
```

Output :



EXERCISE11: REPORTING AND VISUALIZING VARIATION

Reporting and visualizing variation in data involves techniques that help understand how data points vary around a central value or trend. Variation is crucial in statistical analysis as it provides insights into the consistency, stability, or unpredictability of a dataset. Techniques for Reporting and Visualizing Variation involve:

- **Histograms:** Show the distribution of numerical data by grouping data into bins and displaying bars representing the frequency of data points in each bin.
- **Box Plots :** Display the distribution of data based on a five-number summary: minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum. Useful for comparing distributions across categories or group.
- **Control Charts:** Monitor processes over time to detect any unusual variations that might indicate a process is out of control. Common types include X-bar and R charts, which show the central tendency and range or variation over time.
- **Scatter Plots:** Display the relationship between two variables. Useful for identifying correlations and understanding how variation in one variable relates to variation in another.
- **Time Series Plots:** Show data points plotted in chronological order, highlighting trends, seasonality, and other patterns over time.
- **Heatmaps:** Use color gradients to visualize the density or intensity of variation across different categories or dimensions.

Control chart:

The control chart is a graph used to study how a process changes over time. A control chart consists of 4 main features:

- **X and Y-axis Values:** The data value will be the Y-axis. This can be count of customers, count of tickets, revenue, cost, or whatever data value the business wants to measure. Time will be the X-axis.
- **The Center Line (AKA Average):** This is the mean/average of the data points. It is represented with a solid line drawn in the middle section of the graph.
- **Lower Control Limit (LCL):** This is found below the average and is referred to as -3 sigma line, which acts as a lower bound.
- **Upper Control Limit (UCL):** This is above the average and referred to as +3 sigma line, which acts as an upper bound.

So to create the average line, Use this calculation:

Average Oil Variance = `CALCULATE(AVERAGEX(DATA,DATA[OilVariance]),ALLSELECTED(DATA[Date]))`

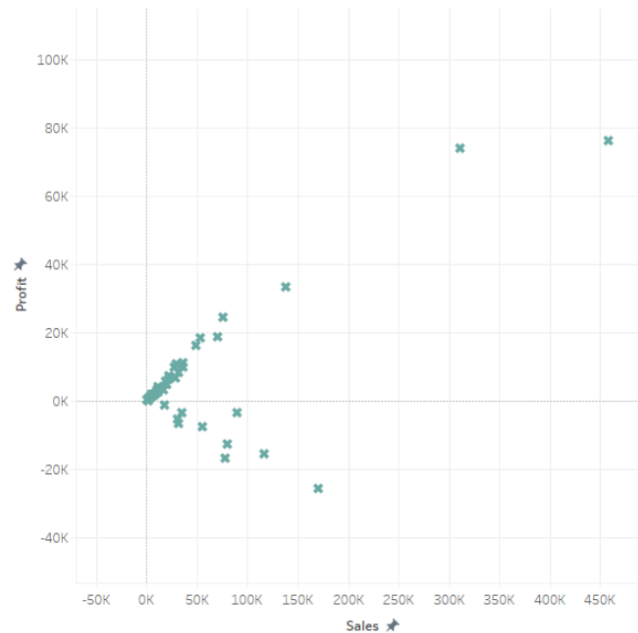
Scatter Plot :

A scatter plot is a type of data visualization that displays the relationship between two continuous variables. It uses Cartesian coordinates to represent the values of the variables.

Key Features of Scatter Plots include:

- **X-Y Axis Representation:**
 - The horizontal axis (X-axis) represents one variable (often the independent variable), while the vertical axis (Y-axis) represents another variable (often the dependent variable).
- **Individual Data Points:**
 - Each data point in the scatter plot represents a single observation or data pair consisting of values from both variables.
- **Pattern Identification:**
 - Scatter plots help in visually identifying patterns or relationships between the two variables. Common patterns include:
 - **Positive Relationship:** As one variable increases, the other variable also tends to increase.
 - **Negative Relationship:** As one variable increases, the other variable tends to decrease.
 - **No Relationship:** No discernible pattern or correlation between the variables.
 - **Non-linear Relationship:** The relationship between variables is not linear but follows some other pattern (e.g., exponential, quadratic).
- **Correlation Assessment:**
 - Scatter plots are useful for assessing the strength and direction of the relationship between variables. Correlation coefficients such as Pearson's r can quantify the degree of correlation observed in the scatter plot.
- **Outlier Detection:**
 - Outliers, which are data points significantly different from others, can be easily identified in scatter plots. They may indicate data quality issues or interesting phenomena.

Scatter Plot



Regression and Trend Lines :

In data visualization, trend lines are graphical representations of the general direction or pattern of a dataset. They are used to visually summarize and highlight trends or patterns within the data. Here are key points about trend lines in data visualization:

Purpose of Trend Lines:

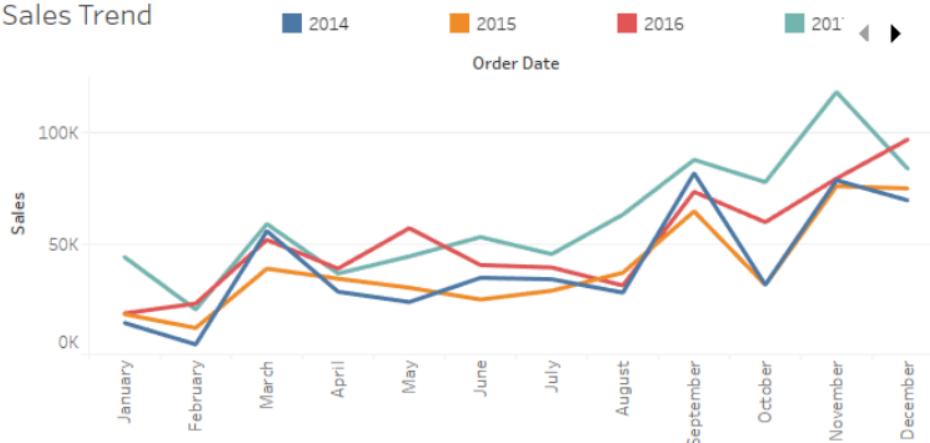
1. Highlighting Trends: Trend lines help to emphasize the general direction (upward, downward, or horizontal) of a dataset over time or across different variables.
2. Pattern Recognition: They aid in identifying patterns such as linear relationships (straight-line trends), exponential growth or decay, quadratic patterns, etc.
3. Forecasting and Prediction: Trend lines can sometimes be used to make predictions about future values based on historical data trends, though caution should be exercised as this depends on the nature and reliability of the underlying data.

Types of Trend Lines:

1. Linear Trend Lines: These are straight lines that best fit the data points in a linear fashion, showing a consistent increase or decrease over time or across variables.
2. Exponential Trend Lines: Exponential trend lines are curved lines that best fit data where the rate of change increases or decreases over time exponentially.
3. Logarithmic Trend Lines: These are curved lines that best fit data where the rate of change decreases over time, typically showing diminishing returns or saturation.
4. Polynomial Trend Lines: These are curves that fit data based on polynomial equations of different degrees (e.g., quadratic, cubic), capturing more complex relationships beyond linear or exponential trends.

Here is a trend line between sales(measure) and order date(measure) :-

Sales Trend



Stacked Bars :

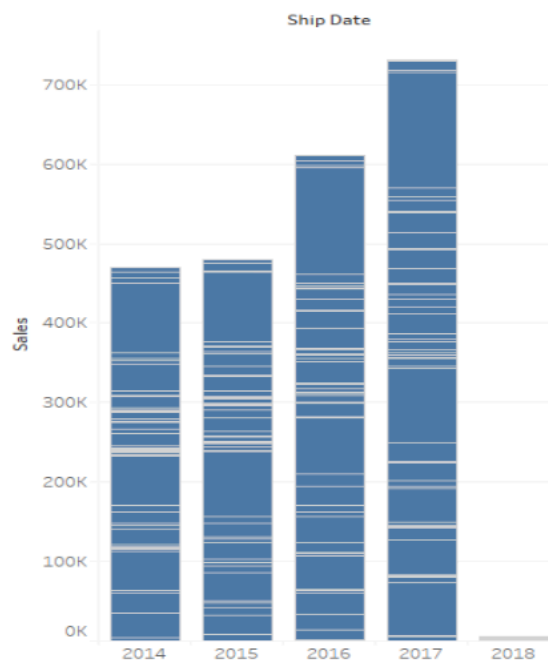
Stacked bars in data visualization are a type of chart where multiple datasets are stacked on top of each other in a single bar. Each segment of the stacked bar represents a part of the whole, and the total bar height (or length, in horizontal stacked bars) represents the sum of all parts.

Key Features and Characteristics of Stacked Bar Charts:

1. **Composition Display:** Stacked bar charts are used to show how individual parts contribute to the whole across different categories or segments. Each bar is divided into segments, with each segment representing a different subgroup or category.
2. **Comparison:** They facilitate comparison not only within categories but also between categories. It allows viewers to easily see how the proportions of different subgroups change across categories.
3. **Part-to-Whole Relationship:** Stacked bars emphasize the relationship between the parts and the whole. Viewers can quickly grasp the total value (whole) and how it is divided into its constituent parts.
4. **Stacking Order:** The order in which segments are stacked can affect interpretation. The segment at the bottom (closest to the baseline) represents the first category or subgroup, while subsequent segments are stacked on top in order.
5. **Absolute vs. Relative Comparisons:** Stacked bars can show absolute values (actual quantities) or relative values (percentages or proportions), depending on how the data is scaled.

Here is a Stacked Bar graph between sales(measure) and ship date (measure) and state(dimension):-

STACKED BAR GRAPH



Exercise 12: Depicting Changes Over Time

Line Chart

A line chart is a graphical representation that displays information as a series of data points called markers connected by straight line segments. It is commonly used to show trends or changes over time, illustrating the relationship between two variables where one variable is plotted on the x-axis (horizontal axis) and the other on the y-axis (vertical axis). Line charts are effective for visualizing continuous data and are particularly useful in analyzing trends and patterns over regular intervals, such as months, years, or quarters.

Dual Axis Line Chart

A dual axis line chart extends the capabilities of a standard line chart by allowing two different sets of data to be plotted on the same chart with separate y-axes that share the same x-axis. This type of chart is beneficial when comparing two variables that have different scales or units of measurement but are related to the same x-axis values (e.g., time periods). By using dual axes, it becomes easier to identify and analyze trends, correlations, and discrepancies between the two datasets.

Scatterplot

A scatterplot is a type of diagram that uses Cartesian coordinates to display values for typically two variables for a set of data. Each dot in the scatterplot represents a single observation where the position on the x-axis and y-axis corresponds to the values of the two variables. Scatterplots are particularly useful for visually identifying relationships or correlations between variables, such as examining how changes in one variable affect another. They are effective in highlighting patterns, clusters, and outliers within the data.

Applications and Relationships to Depicting Changes Over Time

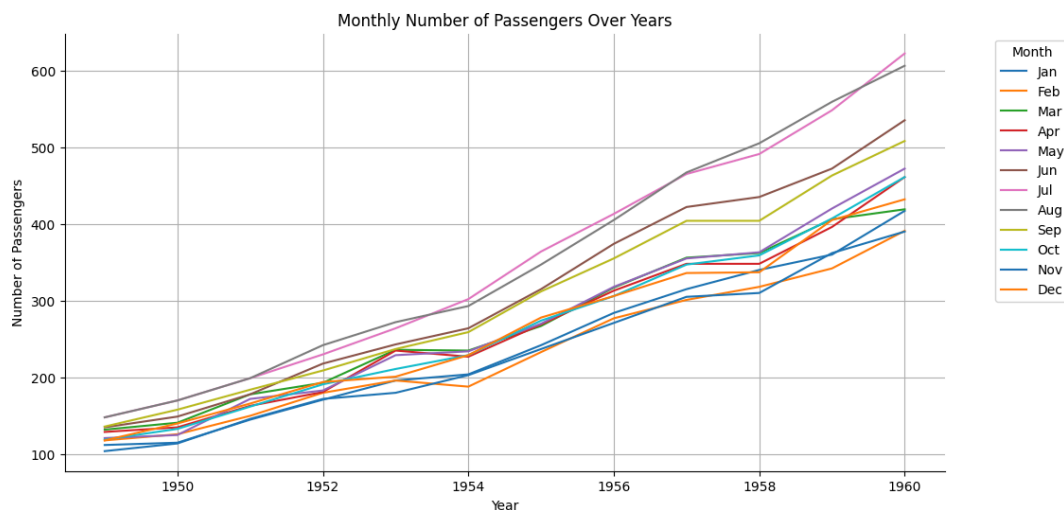
Line Chart: Line charts are ideal for depicting changes over time because they emphasize trends and variations in data points along a continuous timeline. They are commonly used in financial analysis to track stock prices over days, months, or years, in weather forecasting to show temperature changes over seasons, and in business analytics to monitor sales performance across different periods.

Dual Axis Line Chart: This chart type is used when comparing two related datasets that evolve over the same time period but may have different scales or units of measurement. For instance, it can be used in economics to compare GDP growth rates and unemployment rates over the same quarters, allowing analysts to discern potential correlations or discrepancies between the two variables.

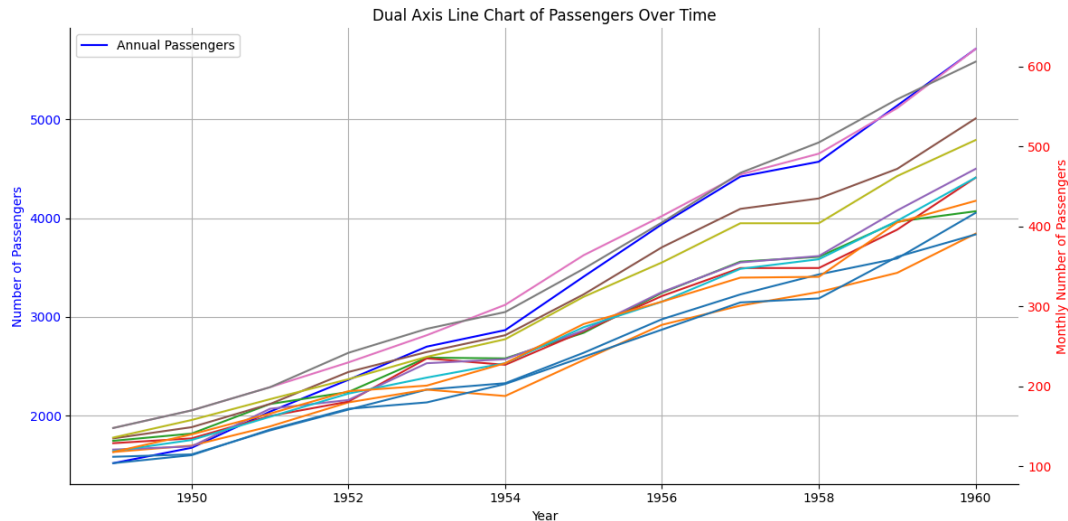
Scatterplot: While not directly depicting changes over time in the traditional sense, scatterplots are valuable for understanding how two variables relate to each other across different observations or instances. In scenarios where time is a factor (e.g., comparing customer satisfaction scores before and after a product launch), scatterplots can help identify patterns of improvement or decline over time.

1. Line Chart Depicting Changes Over Time

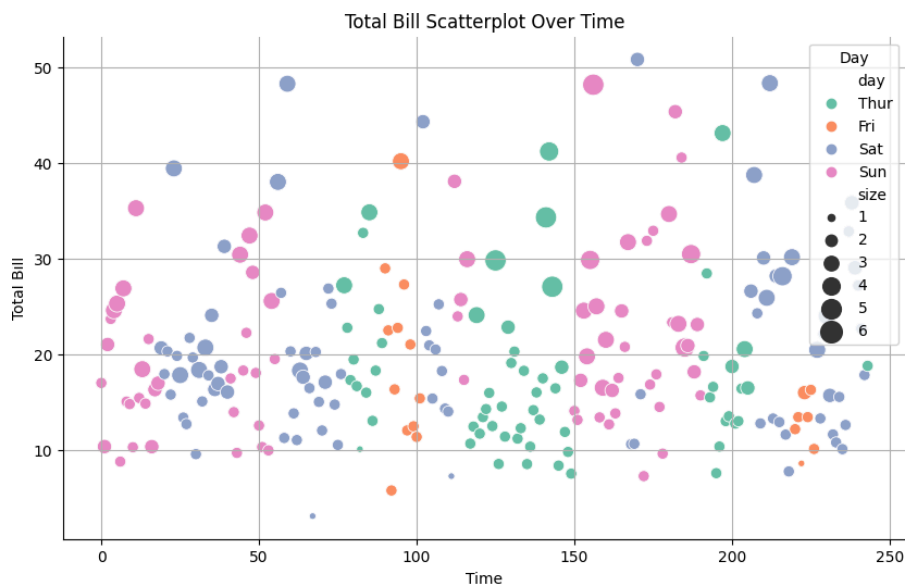
- The x-axis would show the years, and the y-axis would represent the number of passengers.
- Each line would be colored differently to represent different months, as defined by the hue
- Interactive: Users can hover over data points to see specific values (like the number of passengers in a particular month of a year).
- Filtering: Users can filter the data by selecting specific years or months from slicers or filters connected to the chart



The below chart is a dual-axis line chart displaying passenger data over time. The X-axis represents the years from 1949 to 1960. The left Y-axis (blue) shows the annual number of passengers, ranging from 2000 to 5000, while the right Y-axis (red) shows the monthly number of passengers, ranging from -100 to 600. Multiple colored lines represent different data columns. The legend indicates "Annual Passengers" in blue. The chart combines both annual and monthly passenger data over the specified period.



This scatterplot illustrates the relationship between total bill amounts and time, with data points colored by the day of the week (Thursday, Friday, Saturday, Sunday) and sized according to group size (ranging from 1 to 6). The X-axis represents time, while the Y-axis shows the total bill. Different colors and sizes help visualize variations and patterns in the data across different days and group sizes.



- Line Chart: Shows trends over time by connecting data points with lines, ideal for tracking metrics like sales or passenger numbers.
- Dual-Axis Line Chart: Combines two datasets with different scales on one graph, useful for comparing related data (e.g., annual vs. monthly passengers).
- Scatterplot: Displays relationships between variables over time with points, using color and size to add extra dimensions (e.g., total bills by day and group size).

EXERCISE 13: Reporting

DASHBOARD:

A dashboard is an information management tool that receives data from a linked database to provide data visualizations. It typically offers high-level information in one view that end users can use to answer a single question. In many cases, they can be configured to provide specific information to the end user and how this information is visualized. E.g., Numbers, charts, or graphs.

CHARACTERISTICS OF DASHBOARD:

- Consolidated Information:
- Visual Representation
- Interactive Elements
- Real-Time Data
- Performance Monitoring
- Decision Support
- User-Friendly
- Customization

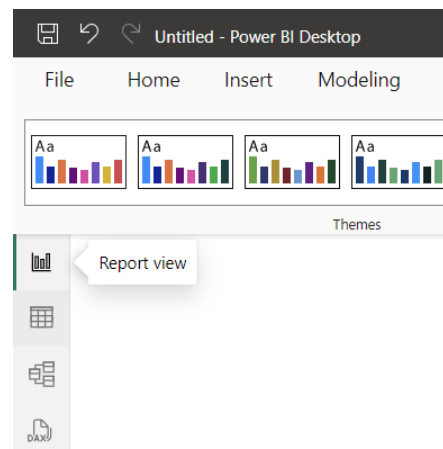
Understanding PowerBI

With Power BI Desktop, you can:

1. **Connect** to data, including multiple data sources (File, Azure, Database, Microsoft fabric, online services etc)
2. **Shape** the data with queries that build insightful, compelling data models.
3. Use the **data models** to create visualizations and reports.
4. Share your report files for others to leverage, build upon, and share. You can share Power BI Desktop .pbix files like any other files, but the most compelling method is to upload them to the Power BI service.

Views in PowerBI

- **Report View:** Design and customize interactive reports and dashboards with various visual elements.
- **Data View:** Inspect and validate the underlying data in a tabular format, and create calculated columns.
- **Model View:** Define and manage relationships between tables to build a structured data model.
- **DAX Query View:** Write and execute DAX queries for advanced data analysis, available through external tools like DAX Studio.



Loading Data related to sales and retail domain:

After importing dataset we have options of load ,transform data:

Extract: Collecting data from various sources.

Transform: Cleaning, formatting, and converting the data into a suitable format.

Load: Storing the transformed data into a target database or data warehouse.

ETL is essential for preparing data for analysis and reporting.

Panes in PowerBI

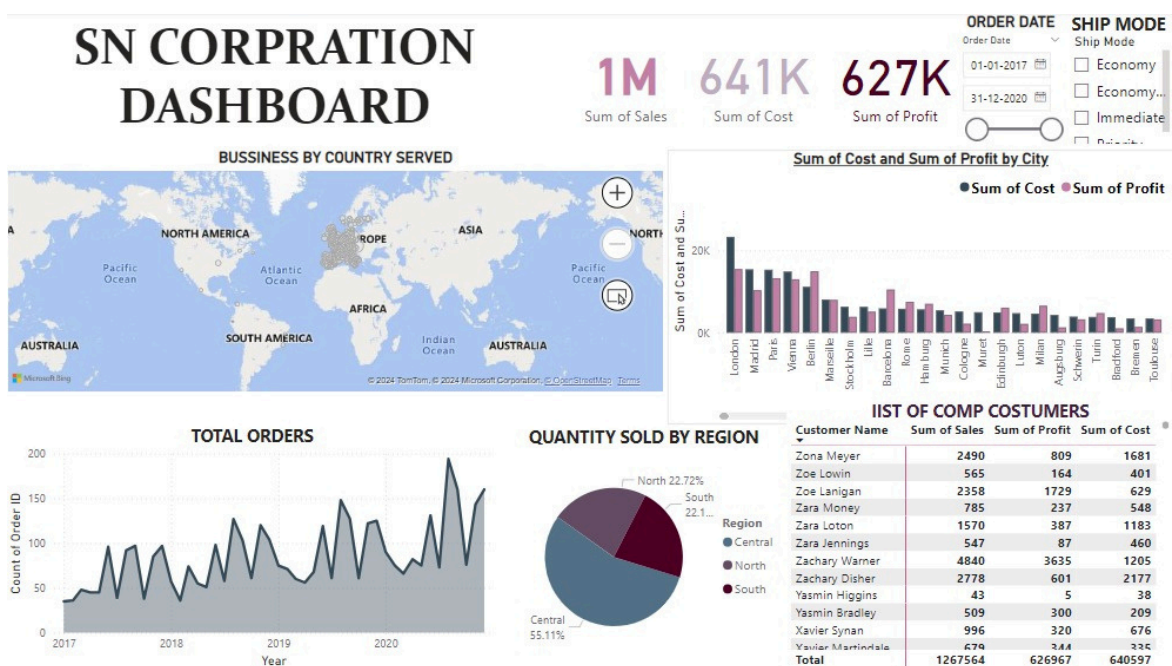
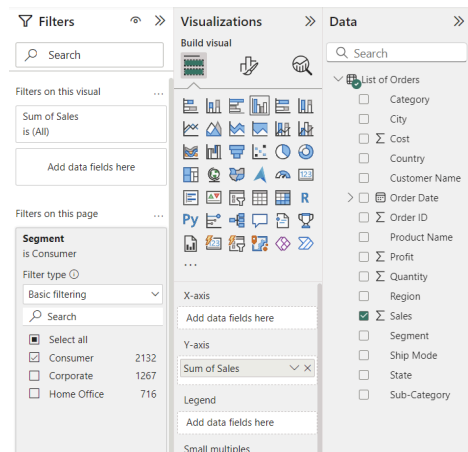
Fields Pane: Displays the datasets available for your report. You can drag fields from this pane to visuals on your report canvas.

Visualizations Pane: Contains various types of charts and visuals that you can add to your report. It also includes options for formatting and customizing these visuals.

Filters Pane: Allows you to apply filters to your data at the visual, page, or report level. This helps in narrowing down the data to focus on specific insights.

Fields Well (within Visualizations Pane): When a visual is selected, this area shows where you can drag fields to configure the visual's axes, values, legends, and other parameters.

Format Pane: Provides options for customizing the appearance of visuals, such as changing colors, fonts, labels, and more.



The SN Corporation dashboard includes various visuals displaying business performance metrics. Here's a concise guide on each visual and how to create them in Power BI:

Map - Business by Country Served:

Description: Shows countries where the corporation operates.

Creation: Use a Map visual and drag the country field to Location.

Card Visuals - Sum of Sales, Cost, Profit:

Description: Displays total sales, cost, and profit.

Creation: Use Card visuals and drag respective measures to Values.

Bar Chart - Cost and Profit by City:

Description: Compares cost and profit across cities.

Creation: Use a Clustered Bar Chart, drag the city field to Axis, and Sum of Cost and Sum of Profit to Values.

Line Chart - Total Orders:

Description: Shows total orders trend over years.

Creation: Use a Line Chart, drag the date field to Axis, and total orders to Values.

Pie Chart - Quantity Sold by Region:

Description: Represents quantity sold by regions.

Creation: Use a Pie Chart, drag the region field to Legend, and quantity sold to Values.

Slicer - Order Date and Ship Mode:

Description: Filters data by order date and ship mode.

Creation: Use Slicer visuals, drag the order date to one slicer and ship mode to another, then adjust settings.

Filters in PowerBI

Filters are essential components of dashboards that enhance usability by allowing users to interactively explore data and customise their view according to specific criteria. Here's how filters are typically used in dashboards

1 Data Exploration : Filters enable users to drill down into specific subsets of data. For example, in a sales dashboard, users might filter data by region, time period, product category, or salesperson to focus on relevant information.

2 Comparative Analysis: Filters allow for side-by-side comparison of different datasets or segments. In a financial dashboard, filters can be used to compare performance across different quarters or compare the performance of different products.

3 Drill-Down Capabilities: Filters facilitate drilling down from high-level summaries to detailed views. For instance, in a project management dashboard, users can filter by project status or team member to see detailed task progress and individual contributions.

4 Dynamic Updates: Filters dynamically update the dashboard visuals based on user selections, providing real-time insights. This is particularly useful in operational dashboards where monitoring live data is crucial.

5 User Personalization: Filters can support personalization, allowing users to save their preferred settings or view presets. This feature enhances user experience by tailoring the dashboard to individual needs.

Storytelling in PowerBI:

Concept: Storytelling in Power BI refers to the ability to create narratives using data insights to convey a compelling message or drive action.

Components: It involves structuring reports and dashboards in a sequence that guides the audience through a story, using visualizations and annotations.

Purpose: Storytelling helps stakeholders understand the significance of data trends, correlations, and predictions in a coherent and engaging manner.

Each dashboard or sheet within the story should build upon the previous one, guiding the viewer through a logical progression of information

